

UNIVERSIDAD POLITÉCNICA DE MADRID

**ESCUELA TÉCNICA SUPERIOR
DE INGENIEROS DE TELECOMUNICACIÓN**



GRADO EN INGENIERÍA BIOMÉDICA

TRABAJO FIN DE GRADO

**DESIGN AND DEVELOPMENT OF A MOTION
CAPTURE LIBRARY FOR VIRTUAL REALITY DEVICES**

**MIGUEL ANGEL DURÁN LUQUE
ENERO 2026**

TRABAJO DE FIN DE GRADO

Título: Diseño y desarrollo de una biblioteca para la captura de movimientos en dispositivos de realidad virtual

Título (inglés): Design and development of a motion capture library for virtual reality devices

Autor: Miguel Angel Durán Luque

Tutor: Rodrigo García Carmona

Departamento: Departamento de Ingeniería de Sistemas Telemáticos

MIEMBROS DEL TRIBUNAL CALIFICADOR

Presidente: —

Vocal: —

Secretario: —

Suplente: —

FECHA DE LECTURA:

CALIFICACIÓN:

UNIVERSIDAD POLITÉCNICA DE MADRID

ESCUELA TÉCNICA SUPERIOR DE
INGENIEROS DE TELECOMUNICACIÓN
Departamento de Ingeniería de Sistemas Telemáticos



TRABAJO FIN DE GRADO

**DESIGN AND DEVELOPMENT OF A MOTION
CAPTURE LIBRARY FOR VIRTUAL REALITY
DEVICES**

Miguel Angel Durán Luque

ENERO 2026

Resumen

El análisis cuantitativo del movimiento humano tiene un papel esencial en diversas áreas de la ingeniería biomédica, en particular en la evaluación clínica, la rehabilitación y el estudio de los trastornos motores. La obtención de datos cinemáticos precisos permite a médicos e investigadores valorar de forma objetiva las limitaciones funcionales, seguir la evolución de los pacientes y diseñar intervenciones terapéuticas a medida. Tradicionalmente, este tipo de análisis se ha sustentado en la captura de movimiento realizada en laboratorios de investigación, dotados de sistemas muy especializados, que a pesar de ofrecer una gran precisión, su elevado coste y limitada disponibilidad dificultan de forma considerable su implementación clínica a gran escala.

En los últimos años, los dispositivos de realidad extendida (XR) han surgido como una alternativa prometedora para capturar y analizar el movimiento humano fuera de los entornos tradicionales de laboratorio. Abriendo nuevas oportunidades para servicios biomédicos, como la rehabilitación en el hogar y la evaluación del comportamiento de los pacientes en entornos virtuales controlados. Sin embargo, estos nuevos sistemas presentan aún desafíos por superar antes de su uso de manera confiable en entornos biomédicos, ya que están enfocados principalmente en la experiencia del usuario y su interacción, más que en la adquisición de datos precisos y temporalmente estables. Además, la mayoría de las aplicaciones de XR carecen de mecanismos para el almacenamiento persistente de datos y la exportación estructurada, lo que limita la reproducibilidad y el posprocesamiento.

En este contexto, el presente trabajo se enmarca en el proyecto LibreMotion, cuyo objetivo es democratizar la captura, el almacenamiento y el análisis de datos biomecánicos mediante tecnologías XR. En concreto, esta tesis se centra en el diseño y desarrollo de LibreMotion Capture, una biblioteca de software multiplataforma destinada a capturar datos de movimiento de la cabeza y las manos desde aplicaciones de realidad virtual (VR, un tipo de XR) y exportarlos en formatos compatibles con los flujos de trabajo biomédicos convencionales. De este modo, se un software robusto y extensible que une los entornos VR interactivos con los estándares biomecánicos establecidos, lo que permite el uso de la VR como una plataforma de detección viable para el análisis del movimiento.

Palabras clave: Ingeniería biomédica, realidad virtual, datos biomecánicos, captura de movimiento, rehabilitación, ingeniería de software.

Abstract

Quantitative analysis of human movement plays a fundamental role in multiple areas of biomedical engineering, particularly in clinical evaluation, rehabilitation, and the study of motor disorders. Accurate kinematic data allow clinicians and researchers to objectively evaluate functional limitations, monitor patient progress, and design personalized therapeutic interventions. Traditionally, this type of analysis has relied on specialized motion capture laboratories equipped with optical systems, force platforms, and dedicated biomechanical instrumentation. Although these systems provide high precision, their high cost, limited accessibility, and constrained environments significantly restrict their widespread clinical adoption.

In recent years, extended reality (XR) technologies have emerged as a promising alternative for capturing and analyzing human motion outside traditional laboratory settings. This opens new opportunities for biomedical applications such as home-based rehabilitation and the assessment of patient behavior in controlled, yet flexible virtual environments.

However, despite the growing interest in XR-based motion analysis, several challenges remain before these technologies can be reliably adopted in biomedical contexts. From an engineering perspective, XR systems are primarily designed for interactive rendering and user experience rather than for precise and temporally stable data acquisition. Moreover, most XR applications lack standardized mechanisms for persistent data storage and structured export, limiting reproducibility and post-processing capabilities.

Within this context, the present work is framed as part of the LibreMotion project, whose objective is to democratize the capture, storage, and analysis of biomechanical data using XR technologies. Specifically, this thesis focuses on the design and development of LibreMotion Capture, a cross-platform software library that aims to capture head and hand motion data from virtual reality (VR, a type of XR) applications and export them to formats compatible with conventional biomedical workflows. Thus, providing a robust and extensible software infrastructure that bridges interactive VR environments with established biomechanical standards, enabling the use of VR as a viable sensing platform for movement analysis.

Keywords: Biomedical engineering, virtual reality, biomechanical data, motion capture, rehabilitation, software engineering.

Agradecimientos

No sería realista ni justo no agradecer este trabajo a mi tutor Rodrigo, por darme la oportunidad de hacer este trabajo tan chulo y aguantarme estos meses. A mi familia, a Paula, a mis compañeros con los que he pasado estos años universitarios, Jesus, Julia, Itziar, Alba, Sara, Alvaro, Miriam, Belen, Marta, y un larguísimo etc.

Pero la irónica realidad es que a la persona a la que quiero agradecer, por todo, la que siempre me ha acompañado, la que más me ha querido, la que se leería todo este trabajo para decir lo maravilloso que es su hijo a pesar de no haber entendido absolutamente nada y la que a su vez se pondría a llorar leyendo esta dedicatoria; es la persona que ya no está aquí para leerla.

Te echamos de menos, mamá.

Contents

Resumen	I
Abstract	III
Agradecimientos	V
Contents	VII
List of Figures	IX
List of Tables	XI
1 Introduction	1
1.1 Context	1
1.2 Objectives	3
1.3 Structure of This Document	4
2 Enabling Technologies	5
2.1 State of the Art	5
2.2 Technologies	7
2.2.1 Meta Quest and Horizon OS	7
2.2.2 Data Tracking via HMD	9
2.2.3 Data Standards	11
2.2.4 Software Development Platforms	14
2.2.5 Native Communication	15
2.2.6 Database Backend	15
3 Requirements	17
3.1 Functional Requirements	17
3.2 Non-functional Requirements	18
4 Design and Implementation	19
4.1 System Architecture	20

4.2	Integration Layer	22
4.2.1	Data Capture	22
4.3	Telemetry Library	23
4.3.1	Telemetry API	24
4.3.2	Custom Data Models	25
4.3.3	Configuration Management	27
4.3.4	C3D Recorder	28
4.3.4.1	Representation of Orientation Data and Timestamp in C3D .	29
4.3.4.2	Additional Points, Coordinate Axes and Measurement Units	30
4.3.5	JSON Uploader	32
5	Validation	37
5.1	Detection and Correction of Data Loss During Capture	37
5.2	Temporal Stability Analysis and Sampling Strategy	40
5.2.1	Specific Considerations in Unreal Engine	43
5.3	Performace Analysis	44
5.4	Validation of Data Persistence Mechanisms	45
5.4.1	Validation of JSON Transmission to Supabase	45
5.4.2	Validation of C3D Session Recording	47
6	Conclusions and Future Work	49
6.1	Future Work	50
	Appendix A Impact of the Project	i
A.1	Social Impact	i
A.2	Ethical and Privacy Considerations	i
A.3	Economic Impact	ii
A.4	Environmental Impact	ii
A.5	Professional Responsibility	iii
	Appendix B Economic Budget	v
B.1	Tecnological Resources	v
B.2	Human resources	v
B.3	Total Cost	vi
	Bibliography	vii

List of Figures

1.1	Position of LibreMotion Capture within the overall LibreMotion framework . . .	2
2.1	Representation of the 6 degrees of freedom	10
2.2	Plug-in Gait upper-body model	13
2.3	OpenXr Hand joints	14
4.1	General architecture diagram	21
4.2	Head markers	31
4.3	Wrist markers	31
5.1	Missing frames on simple movement	38
5.2	Similar movement with optimized processing	39
5.3	X-Z circular trajectory - 2 sec interval	41
5.4	Jitter comparison across callbacks	42
5.5	CPU utilization across 4 different test	45
5.6	Screenshot from Supabase web server tool showing stored frames	47
5.7	Unity Screenshot	48
5.8	C3D-Visualizer	48
5.9	UE Screenshot	48
5.10	Mokka C3D Visualizer	48

List of Tables

4.1	Public methods exposed by the telemetry API and their role within the application lifecycle.	24
4.2	Main functions in <code>configReader</code> and their role within the application lifecycle	27
4.3	Configurable recording parameters	28
4.4	Public methods in <code>C3DRecorder</code> and their role within the application lifecycle	29
4.5	Public methods in <code>JSONUploader</code> and their role within the application lifecycle	33
4.6	Public methods in <code>TelemetryManager</code> and their role within the application lifecycle	34
4.7	Main methods in <code>HTTPHelper</code> and their role within the telemetry upload lifecycle	35
5.1	Temporal jitter analysis for different pose sampling callbacks.	42
B.1	Estimated total cost of the project	vi

Introduction

1.1 Context

Quantitative analysis of human movement plays a fundamental role in many areas of medicine, especially in fields such as rehabilitation, physiotherapy, neurology and clinical biomechanics. Accurate monitoring of a patient's posture, orientation, and body dynamics allows objective assessment of the progression of musculoskeletal pathologies, analysis of gait patterns, detection of motor compensation, and design of personalized treatments based on measurable and reproducible data. Traditionally, this type of analysis has been performed using specialized motion capture systems, which offer high precision but have significant limitations in terms of cost, installation complexity, and accessibility.

In recent years, the popularization of virtual reality (VR) devices has opened up new opportunities for motion capture in uncontrolled environments. Due to their relatively low cost and growing adoption, virtual reality headsets are a promising alternative for democratizing biomechanical data capture and transferring these technologies from specialized laboratories to clinical, domestic, or tele-rehabilitation environments.

However, the use of these devices for medical and scientific purposes poses significant challenges. The data generated by VR systems is typically geared towards the development of interactive applications or video games, rather than storage, analysis, or reuse in biomedical contexts. Furthermore, traditional formats used in biomechanics, such as Coordinate

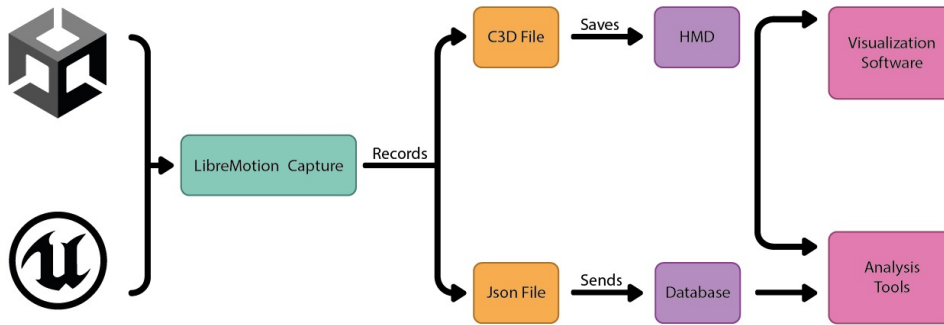


Figure 1.1: Position of LibreMotion Capture within the overall LibreMotion framework

3D (C3D), were designed for classic capture systems and are not natively adapted to the characteristics of movement in VR, such as the extensive use of orientations represented by quaternions or the need for real-time data transmission. Added to this is the fragmentation of the hardware and software ecosystem, as well as the absence of open solutions that facilitate the capture, storage, and visualization of these data in a standardized manner.

This is the context in which LibreMotion emerged, an open-source project whose goal is to democratize the capture, storage, and analysis of biomechanical data obtained from XR (extended reality) devices. LibreMotion is conceived as a modular ecosystem, as shown in Figure 1.1 consisting of an XR-oriented motion capture format and library, an open and privacy-friendly database to store anonymized data sets, and a web-based visualization tool that allows data to be inspected without the need for proprietary software. The project aims to serve as a bridge between real-time tracking data provided by standards such as OpenXR and the rigor, precision, and contextualization requirements specific to the biomedical field.

This Bachelor Thesis is part of this global project and focuses specifically on the development of LibreMotion_Capture, the library responsible for motion capture. The aim of this library is to facilitate the integration of biomechanical data acquisition into existing XR applications, enabling efficient and customizable recording of the positions and orientations of various elements tracked by virtual reality devices. The solution is also intended to be multi-platform, reusable, and easily integrated into widely used graphics engines, so that it can be adopted by developers, researchers, and health professionals alike.

In this way, this work contributes to the technical foundations necessary for the use of extended reality (and, more particularly, virtual reality) devices as motion capture tools in medical and scientific contexts, aligning with the philosophy of openness, interoperability, and accessibility that defines the LibreMotion project.

1.2 Objectives

The main objective of this Bachelor Thesis is to design and develop a multi-platform software library for motion capture in virtual reality applications, capable of accurately and systematically recording the position and orientation of the user's head and hands and storing these data in formats suitable for subsequent biomechanical and scientific analysis.

This library must be easily integrated into applications developed with the most widely used game engines in the field of virtual reality, ensuring interoperability between environments and the reuse of the system in different projects and research contexts.

In order to achieve the overall objective, the following specific objectives have been defined:

- Design a common and consistent API that allows access to motion capture functionality from different development environments, including applications based on Unity, Unreal Engine, and native code, while maintaining a unified and consistent interface.
- Implement motion data capture from the main elements tracked by virtual reality devices, including the head-mounted display (HMD), hand controllers, and, when available, hand tracking.
- Standardize coordinate and orientation systems, ensuring consistency of captured data regardless of the graphics engine or internal reference system used by each platform, so that results are comparable and reusable.
- Manage telemetry in a configurable manner, allowing key operating parameters—such as sampling frequency, feature activation, or storage destinations—to be defined using an initial configuration file external to the application.
- Implement mechanisms for the persistence of captured data, including:
 - Create C3D files, with the aim of maintaining compatibility with traditional biomechanical analysis tools.
 - Send data via HTTP POST requests to an external backend in JSON format, facilitating remote storage and integration with databases or web services.
- Facilitate the integration of the library into third-party projects, minimizing configuration requirements and reducing the complexity involved in incorporating motion capture into existing applications.
- Validate the correct functioning of the developed system through practical tests on real virtual reality devices, checking the consistency and quality of the generated data.

1.3 Structure of This Document

This document is structured as follows.

Chapter 1 introduces the context of the project, outlines the motivation behind the work, and defines the main objectives to be achieved.

Chapter 2 presents the background and enabling technologies required to understand the technological framework in which the project is developed, as well as the relevant state of the art.

Chapter 3 defines the functional and non-functional requirements of the system, establishing the scope and constraints of the proposed solution.

Chapter 4 spans the largest part of the work given the nature of this project. It describes the design and implementation of the developed library, detailing the system architecture, data model, coordinate system, and integration with different game engines.

Chapter 5 presents the experimental results and the validation of the system, analyzing its behavior and performance in real usage scenarios.

Chapter 6 summarizes the main conclusions of the project and discusses possible directions for future work. Additional information related to the project, such as impact analysis and economic considerations, is included in the **appendices**.

Enabling Technologies

2.1 State of the Art

For decades, motion capture has been a key discipline in fields such as biomechanics, digital animation, robotics, and sports medicine. In both research laboratories and professional animation studios, optical motion capture systems based on infrared cameras have long been regarded as the gold standard for human motion analysis. These systems operate by surrounding a capture volume with multiple infrared cameras that track either passive markers—spheres coated with retroreflective material—or active markers that emit light in a synchronized manner. Markers are placed at predefined anatomical locations on a suit worn by the subject. Due to their high spatial and temporal resolution, optical systems can achieve sub-millimeter accuracy at sampling frequencies of up to 500 Hz [1]. This level of precision and reliability has consolidated commercial solutions such as Vicon, OptiTrack, and Qualisys as reference technologies in laboratories, film production, and video game development pipelines [2].

Despite their performance, these systems have limitations, mainly the high cost (over \$100,000) of installing a complete system [3], the technical expertise needed to set it up, and the limited or non-existent portability of the devices, which has motivated the search for more accessible alternatives.

These limitations are particularly relevant in several areas of biomedical engineering,

where motion capture plays a central role in applications such as gait analysis, rehabilitation assessment, motor disorder evaluation, and remote patient monitoring. In these contexts, the high cost, the need for technical (instead of medical) expertise, the lack of portability, and the infrastructure requirements of optical motion capture systems have motivated the adoption of more accessible alternatives. Among them, inertial measurement units (IMUs), based on accelerometers, gyroscopes, and magnetometers, are commonly used to estimate body segment orientation and movement, while RGB-D (Red, Green, Blue, plus Depth) cameras, such as Microsoft Kinect, combine color and depth information with computer vision techniques to reconstruct a three-dimensional human skeleton. Due to their reduced cost, ease of deployment and use, and portability, these technologies have partially democratized motion capture in clinical and research settings, although at the expense of significantly lower accuracy compared to optical systems [4]. Overall, these intermediate solutions represent a compromise between measurement precision and practical usability in biomedical applications.

In line with the evolution toward more accessible motion capture solutions described above, virtual reality technologies have followed a similar trajectory. Although the first VR head-mounted displays were developed in the late 1980s, early systems were prohibitively expensive and largely confined to laboratory environments or highly specialized industrial applications because of their complexity and limited deployability. A major turning point occurred in 2012 with the release of the Oculus Rift DK1, which introduced a head tracking system that combined inertial sensors with external cameras, significantly improving the tracking capabilities while targeting a broader audience [5].

From then on, consumer-oriented VR platforms began to emerge, offering not only immersive visual experiences but also increasingly reliable tracking of head and upper-limb kinematics. This capability has positioned modern VR devices as a promising tool for the acquisition of biomechanical data in functional and ecologically valid scenarios, enabling the evaluation of joint kinematics such as elbow and cervical motion during natural movements [6, 7]. Moreover, the high temporal density and continuity of motion data captured during functional tasks have been shown to provide valuable indicators to distinguish between healthy and pathological movement patterns [8].

This technological shift has fostered the rapid growth of a dedicated VR ecosystem, attracting major companies such as Sony, Apple, HTC, and PICO through the development of proprietary hardware and software platforms. Within this competitive landscape, Meta has consolidated its position as the dominant market player, with its Quest devices accounting for an estimated 50% to 70% of the global VR headset market [9].

Advances in tracking technology have led VR manufacturers to adopt different approaches for estimating the position and orientation of HMDs. Early consumer VR sys-

tems commonly relied on outside-in or beacon-based tracking, in which the user’s motion is captured by external sensors or base stations placed around the interaction space. While this configuration can provide high tracking accuracy, it requires a dedicated setup, precise sensor placement, and calibration, limiting its portability and ease of deployment.

In contrast, more recent devices have increasingly adopted inside-out tracking systems, where onboard cameras and inertial sensors are used to estimate the device pose directly from the environment. These systems rely on computer vision algorithms, typically based on simultaneous localization and mapping (SLAM) and visual-inertial sensor fusion, to continuously track motion without external references [10]. By eliminating the need for external tracking infrastructure, inside-out tracking significantly improves portability and reduces installation complexity, making VR systems more suitable for use outside controlled laboratory environments.

The rapid expansion of the VR ecosystem, driven by the coexistence of multiple hardware manufacturers and tracking architectures, led to a highly fragmented development landscape. This fragmentation was further reinforced by the distinction between PC-based VR systems, which rely on external computing resources and often outside-in tracking setups, and standalone VR devices, which integrate processing, sensing, and tracking capabilities directly within the headset and predominantly rely on inside-out tracking. As a result, application developers were required to target device-specific software development kits (SDKs), increasing development complexity and limiting portability across platforms.

In response to this challenge, the OpenXR standard, promoted by the Khronos Group, was introduced with the goal of providing a unified and vendor-agnostic application programming interface for extended reality (XR) devices [11]. OpenXR defines a common abstraction layer that enables access to core XR functionalities—such as head-mounted display tracking, controllers, and hand tracking—independently of the underlying hardware or operating system. Consequently, major development engines such as Unity and Unreal Engine can interface with a wide range of PC-based and standalone VR devices through a shared API, significantly simplifying cross-platform development and deployment.

2.2 Technologies

2.2.1 Meta Quest and Horizon OS

Head-mounted displays (HMDs) are devices that enable users to perceive and interact with digitally generated content through immersive visual interfaces. Depending on how digital content relates to the physical environment, these devices can support different modalities within the broader concept of extended reality. Virtual reality (VR) refers to experiences

in which the user is fully immersed in a synthetic digital environment that replaces sensory input from the real world, whereas mixed reality (MR) combines real world perception with virtual elements that are spatially anchored to the physical environment and can interact with it. Both paradigms coexist within modern XR headsets and share common sensing and tracking technologies.

From a hardware perspective, VR headsets typically consist of a central unit—the head-mounted display—which is worn on the head and integrates one or more displays acting as a portable visual interface, together with a set of cameras and inertial sensors used to estimate the user’s position and orientation in space [12]. Interaction is traditionally supported through handheld controllers equipped with position tracking, buttons, and analog triggers and joysticks. However, recent consumer devices incorporate camera-based hand tracking and advanced computer vision algorithms capable of estimating hand pose and finger articulation without the need for external markers or physical controllers, positioning these systems as potential tools for motion capture and biomechanical data acquisition.

For this work, the Meta Quest 2 device was selected. This choice is motivated not only by the widespread adoption of Meta headsets in the consumer VR market [9], but also by their strong developer ecosystem, including official SDKs for major development engines such as Unity and Unreal Engine, native support for the OpenXR standard, and the integration of high-quality visual-inertial sensors within a relatively low-cost standalone platform.

From this point onward, all technical descriptions and software examples will refer specifically to Meta Quest 2. Although several concepts related to inside-out tracking, sensor fusion, and application architecture are shared among other standalone Android-based XR devices, they are not directly transferable to fundamentally different platforms such as PC-based VR systems or proprietary ecosystems such as Apple’s XR devices, which are based on distinct hardware architectures, operating systems, and development frameworks.

At the software level, Meta Quest devices run on Horizon OS, an operating system developed by Meta and built on top of the Android Open Source Project, specifically adapted for standalone VR devices. Horizon OS provides an execution environment optimized for real-time graphics, low-latency input processing, and access to tracking subsystems, while also offering native compatibility with the OpenXR standard [13].

Despite its Android foundation, Horizon OS is not completely open source. Developers interact with the system exclusively through documented APIs, official SDKs, and the OpenXR runtime, without the possibility of inspecting, modifying, or replacing internal operating system components. This architectural decision has important implications for the scope of this work and must be carefully considered. First, direct access to raw sensor data is not permitted: tracking information exposed to applications is already processed by proprietary pipelines that typically include sensor fusion, filtering, interpolation, and prediction

algorithms whose internal behavior is not publicly documented. As a result, developers must treat tracking data as high-level estimates rather than raw measurements, which introduces inherent limitations for precise biomechanical analysis.

Second, Horizon OS enforces strict performance and energy management policies designed to preserve the thermal stability and battery life of HMDs. These constraints include dynamic CPU (central processing unit) and GPU (graphics processing unit) frequency throttling based on temperature, limitations on sustained high-load background processes, and explicit restrictions on the execution of background applications [14]. Such limitations directly affect the design of software systems intended for continuous data acquisition, long recording sessions, or real-time processing pipelines.

During the initial planning stage of this project, these operating system constraints led to a significant revision of the original design approach. The initial objective was to develop a standalone background application capable of continuously recording tracking data while other VR applications were running in the foreground. However, this approach proved infeasible due to the restrictions imposed by Horizon OS and the denial of special permissions required to deploy persistent background services, even after formally justifying the academic and non-commercial nature of the project.

Consequently, the project shifted toward the development of a lightweight and modular software library designed to be directly integrated into third-party virtual reality applications during their development process. This design choice ensures compliance with the operating system’s execution model while still enabling systematic access to tracking data, and significantly increases the practical applicability of the proposed solution for developers working within the constraints of commercial standalone VR platforms.

2.2.2 Data Tracking via HMD

Data tracking through HMDs involves the continuous acquisition of user kinematic variables through the set of sensors integrated into the device. These variables include both translational and rotational components of motion, which are estimated by combining data from inertial and visual sensing subsystems. Rather than exposing raw sensor signals, modern standalone VR devices provide developers with pose estimates that result from internal sensor fusion pipelines.

The Meta Quest 2 integrates a multi-sensor hardware architecture designed to support six degrees of freedom (6DoF) tracking of the head as represented on 2.1 [15]. In this context, 6DoF refers to the ability to estimate three translational degrees of freedom—movement along the x, y, and z axes and three rotational degrees of freedom—roll, pitch, and yaw, allowing the full spatial pose of the device to be determined at any instant.

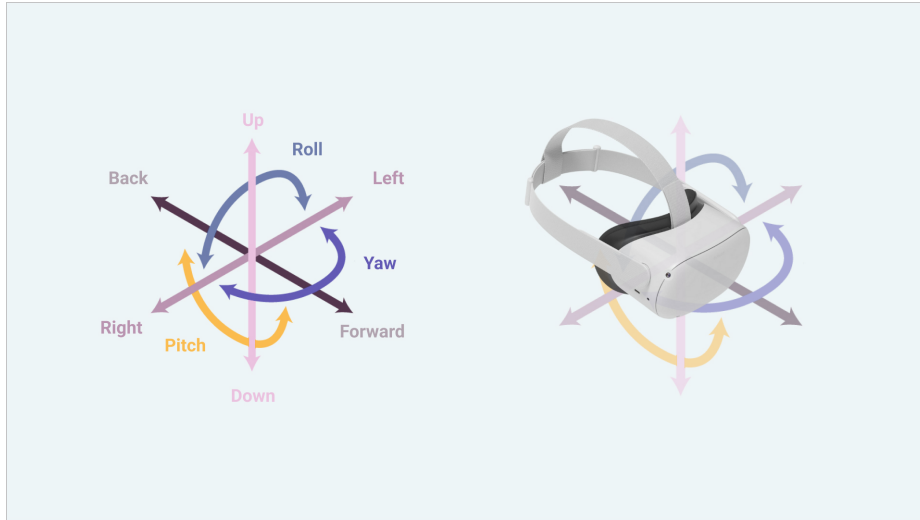


Figure 2.1: Representation of the 6 degrees of freedom

At the inertial level, the device incorporates a single IMU that provides six inertial channels: a triaxial accelerometer with a measurement range of ± 16 g and a triaxial gyroscope with a range of ± 2000 °/s. These sensors are integrated within the Qualcomm Snapdragon XR2 platform and operate at high sampling rates to capture rapid head movements with low latency [16, 17]. The accelerometer measures linear acceleration, while the gyroscope captures angular velocity, both of which are essential for short-term motion estimation and orientation tracking.

In addition to inertial sensing, the Meta Quest 2 relies on multiple outward-facing infrared cameras distributed around the headset to enable inside-out visual tracking. These cameras continuously observe the surrounding environment and extract visual features that are used by simultaneous localization and mapping SLAM algorithms to estimate the position of the headset relative to the physical space. By combining visual information with inertial measurements through visual-inertial sensor fusion, the system compensates for drift inherent to IMU-based integration while maintaining robustness during fast or complex movements.

Despite the advances in visual-inertial tracking systems, motion capture based on head-mounted displays is subject to several inherent limitations that must be considered when using these devices for biomechanical data acquisition. One of the most relevant challenges is the presence of occlusions, which occur when parts of the environment or the user's own body temporarily block the field of view of outward-facing cameras. During these events, visual tracking becomes unreliable or unavailable, forcing the system to rely predominantly on inertial estimation, which can introduce drift or reduced positional accuracy until visual features are recovered.

Another important limitation is temporal aliasing, which arises when the sampling frequency of the tracking system is insufficient to accurately capture rapid movements. Although modern HMDs operate at relatively high update rates, fast head or hand motions may still exceed the effective bandwidth of the sensing and estimation pipeline, leading to distorted motion trajectories or underestimation of peak velocities. This effect is particularly relevant in functional or expressive movements that could be the subject of biomechanical studies.

Furthermore, tracking data obtained from VR devices often exhibit jitter, characterized by small, high-frequency fluctuations in the estimated position or orientation. Jitter can result from sensor noise, numerical instability in sensor fusion algorithms, or rapid switching between visual and inertial dominance within the tracking pipeline.

Finally, sampling rate variability represents an additional constraint. Although tracking systems are designed to operate at a nominal update frequency, real-world conditions such as processing load, thermal throttling, or internal scheduling policies can lead to irregular time intervals between consecutive samples. This variability complicates time-based analysis and requires careful handling of timestamps to avoid inaccuracies in temporal alignment and signal processing.

2.2.3 Data Standards

The design of a motion capture system intended for biomechanical analysis and modern XR applications requires the coexistence of heterogeneous standards originating from different technological domains. On the one hand, extended reality devices rely on real-time tracking interfaces designed for interactive applications. On the other hand, biomechanical and clinical research traditionally depends on well-established data formats optimized for precision, reproducibility, and post-processing. In this project, these two perspectives are represented by the OpenXR standard and the C3D format, respectively.

OpenXR is an open application programming interface promoted by the Khronos Group with the objective of providing a unified way to access XR hardware capabilities. Rather than defining a data storage or exchange format, OpenXR focuses on exposing real-time tracking information from head-mounted displays, controllers, and hand-tracking systems through a common abstraction layer. At its core, OpenXR represents motion data in terms of poses, each defined by a position vector and an orientation expressed as a quaternion. This representation is particularly well suited for XR applications, as quaternion-based rotations avoid singularities and discontinuities associated with Euler angles, and allow smooth interpolation of orientations during real-time rendering.

Poses are typically expressed relative to a defined reference space, a local tracking space

or a world-aligned coordinate system, depending on the application configuration. Tracking space refers to a coordinate system anchored to the physical environment and continuously updated by the device’s tracking pipeline, while world space is usually defined by the application or game engine map and can include additional transformations.

Despite its flexibility and real-time capabilities, OpenXR does not provide mechanisms for recording complete sessions, storing time series data, or exchanging motion datasets between applications. It is explicitly designed as a hardware access API, leaving data persistence, synchronization, and semantic interpretation to the application layer.

In contrast, the C3D format has long been considered the industry standard for biomechanical motion analysis, particularly in gait analysis, rehabilitation, and clinical research. C3D files are designed to store time series of kinematic and kinetic data together with rich metadata, including sampling frequency, subject information, marker labels, and experimental parameters. This structure ensures reproducibility and compatibility with a wide range of biomechanical analysis tools.

However, C3D is fundamentally a file-based and offline format, conceived for optical marker-based motion capture systems operating at fixed and stable sampling frequencies. In particular, the C3D data model assumes a single global sampling frequency defined at the file level, from which the timestamp of each frame is implicitly derived. This design implies that all samples are assumed to be equally spaced in time, and it does not provide a native mechanism to represent variable inter-sample intervals or per-frame timing information. While this assumption is well suited for traditional optical motion capture systems operating under controlled laboratory conditions, it becomes a significant limitation when dealing with data acquired from XR devices, where the effective sampling rate may fluctuate due to rendering load, system scheduling, or internal prediction mechanisms.

C3D neither natively supports real-time streaming, web-based data exchange, or incremental recording during execution. Moreover, the format was originally designed around three-dimensional point trajectories and does not provide native support for quaternion-based joint orientations, which are intrinsic to XR tracking systems. As a result, adapting C3D to VR-derived motion data requires extensions to the data model, such as the inclusion of additional channels for orientation information and explicit handling of coordinate system conventions.

Despite these limitations, C3D was selected in this project due to its widespread adoption in biomechanical research, its well-defined structure, and its compatibility with existing clinical workflows. Furthermore, the format allows for the incorporation of custom labels and metadata, enabling extensions aligned with gait models and facilitating interoperability with established analysis pipelines.

The approach adopted in this project aims to bridge this gap by combining the strengths

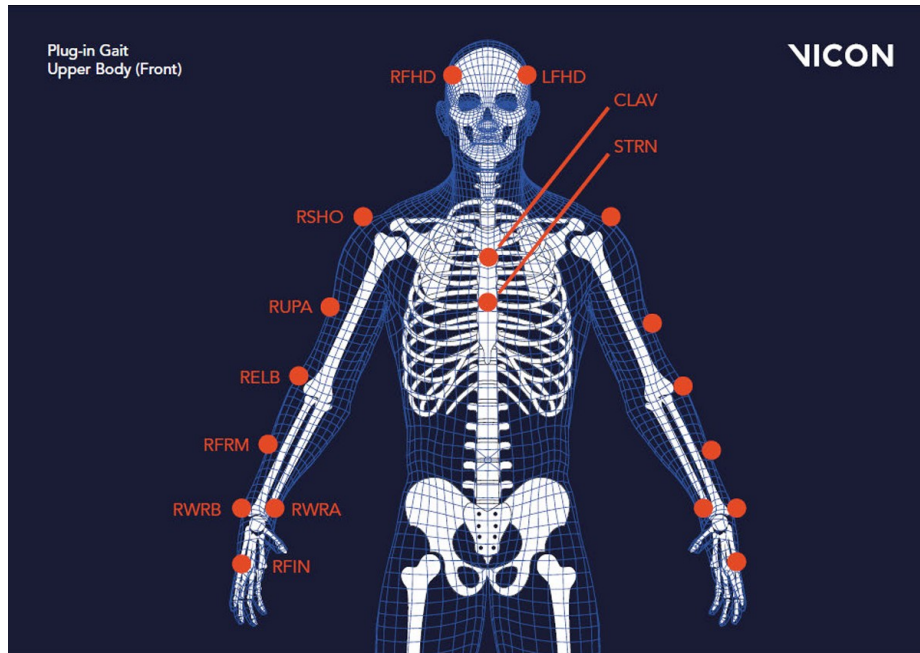


Figure 2.2: Plug-in Gait upper-body model

of both standards. Real-time pose data are acquired through OpenXR using quaternion-based representations and well-defined reference spaces, while recorded datasets are exported using an extended C3D structure adapted to VR tracking characteristics. This design enables compatibility with biomechanical analysis tools while accommodating the temporal variability, coordinate system considerations, and orientation data intrinsic to XR devices. Additionally, this hybrid approach provides a foundation for future extensions, including web-based data pipelines and privacy-aware data handling, which are increasingly relevant in modern clinical and research contexts.

Beyond the definition of data formats and interfaces, biomechanical motion analysis is based on reference models that define how anatomical landmarks and joint kinematics are represented. In the context of C3D-based workflows, one of the most widely adopted reference models is Plug-in Gait [18], created by *Vicon*, which has become a standard in clinical gait analysis and rehabilitation studies. This model, as shown in figure 2.2 [18] defines a fixed set of virtual markers associated with the main body segments and joints, enabling the computation of kinematic variables such as joint angles or trajectories in a consistent and reproducible manner. In contrast, the OpenXR hand joint shown in figure 2.3 [11] model enables the capture of detailed hand pose and finger movement, offering a level of granularity that goes well beyond what is traditionally represented in Plug-in Gait-based C3D datasets.

As a result, a direct one-to-one mapping between OpenXR hand joints and the Plug-in

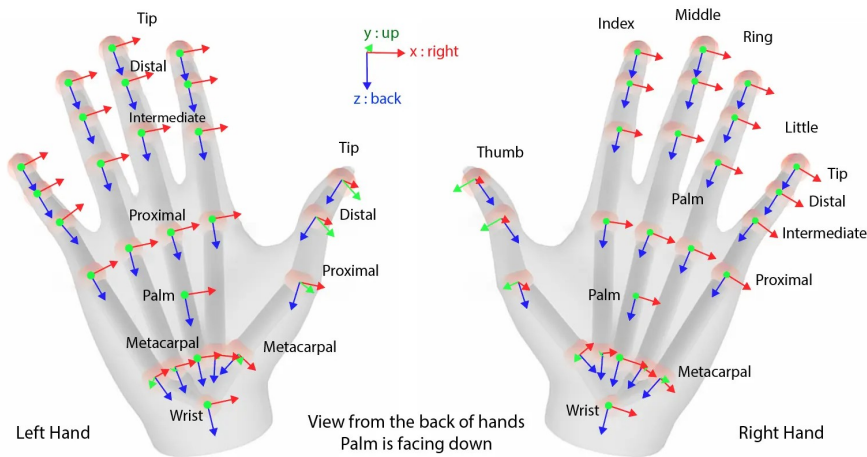


Figure 2.3: OpenXR Hand joints

Gait marker set is not feasible. Instead, this project adopts a hybrid approach in which the core body representation remains compatible with Plug-in Gait conventions, ensuring interoperability with established biomechanical analysis tools, while additional hand-specific data derived from the OpenXR joint model is incorporated as an extension to the standard C3D structure.

2.2.4 Software Development Platforms

The development and integration of motion capture systems for XR applications is strongly conditioned by the software platforms used to build and execute the applications. In the current XR ecosystem, the two dominant game engines are Unity and Unreal Engine (UE) [19], both of which provide extensive support for VR development, native integration with OpenXR, and deployment pipelines targeting standalone and PC-based HMDs.

Both engines implement complex real-time execution models based on iterative update loops, commonly referred to as engine life cycles. These life cycles define when input data is processed, when tracking information is updated, when application logic is executed, and when rendering commands are submitted to the graphics pipeline. As a consequence, the temporal point at which a pose is queried from the XR runtime directly affects the accuracy, latency, and consistency of the captured motion data.

From a motion capture perspective, this aspect is particularly relevant because game engines may internally apply techniques such as late latching or prediction to minimize perceived latency in rendered frames. These mechanisms can lead to discrepancies between the pose used for visual rendering and the pose obtained by the application logic, especially when poses are sampled at different stages of the frame pipeline. In addition, both engines rely on internal timing systems and clocks that may differ from the physical sampling rates of

the underlying tracking hardware, which introduces additional considerations when aligning timestamps for biomechanical analysis.

The choice of when and how poses are sampled must account for engine-specific update phases, synchronization with the XR runtime, and the relationship between rendered and logical poses. Therefore, a detailed analysis of these aspects, including an evaluation of the most suitable sampling points within the life cycle of each engine, is addressed in later sections of this document.

2.2.5 Native Communication

Although Meta Quest devices run on an Android-based operating system, XR applications deployed on these platforms frequently rely on native code to meet performance, portability, and integration requirements. In this project, the core motion capture logic is implemented in C++, which allows direct compilation to native binaries using the Android Native Development Kit (NDK) and facilitates integration with multiple execution environments, including Unity and Unreal Engine.

The Android NDK enables the use of C and C++ for performance-critical components and for sharing a common codebase across different engines and platforms. Native code is compiled using standard toolchains, typically orchestrated through CMake, which provides a portable and reproducible build configuration to generate shared libraries (*.so files). This approach is particularly well suited for XR applications, where real-time constraints and deterministic behavior are essential for motion capture and signal processing tasks.

To integrate native libraries into Android-based XR applications, the compiled native code is packaged within an Android Archive (AAR). The AAR format serves as a container that bundles native binaries together with Java classes, metadata, and configuration files, allowing the library to be easily imported into Unity or Unreal Engine projects targeting Android. This packaging strategy enables a clean separation between engine-level code and platform-specific functionality while preserving a unified deployment pipeline.

2.2.6 Database Backend

In this work, Supabase has been used as the backend storing platform. Supabase is an open-source backend-as-a-service solution that provides database storage, authentication, and data access through a unified interface. While internally based on a relational database, Supabase exposes its data access layer primarily through a Representational State Transfer (RESTful) API that accepts and returns structured data in JavaScript Object Notation (JSON) format.

This architectural choice has important implications for the design of the motion capture

pipeline. In traditional database-driven systems, developers are required to define explicit database schemas, manage table structures, and construct SQL queries for data insertion and retrieval. In contrast, Supabase allows applications to interact with the backend by submitting JSON payloads over HyperText Transfer Protocol (HTTP), abstracting away direct interaction with Structured query language (SQL) and database schemas. As a result, the VR application and the native telemetry library are not required to generate SQL commands or maintain a tight coupling with a predefined relational schema.

This approach enables flexible data structures that can evolve over time as new tracking features or metadata fields are introduced, without requiring changes to the application-side logic. Finally, it aligns naturally with web-based analysis pipelines, where JSON has become the de facto standard for data.

At a higher level, communication between the VR application and external services—such as data storage endpoints—is commonly performed using the HTTP protocol, which provides a standardized and widely supported mechanism for transmitting structured data over the network. In this work, HTTP-based data transmission was intentionally implemented in Java rather than native C++. This design choice leverages Android’s mature networking stack, official APIs, and security mechanisms, including managed threading, Transport Layer Security (TLS) support, and platform-level permission handling. Implementing HTTP communication in Java also reduces complexity compared to native networking solutions, which would require additional third-party libraries and platform-specific adaptations when using C++.

Requirements

As mentioned in the Introduction, the work we have carried out is part of a larger project that is yet to be fully developed. Therefore, the architecture, operation, and development of this part of the Libremotion project must meet certain criteria to ensure proper performance and adaptation to the other parts. However, before addressing the design and implementation of the system, it is necessary to precisely define the requirements that the library and extensions developed must meet. These requirements help define the boundaries of the project's scope, establish objective validation criteria, and ensure that the proposed solution meets the functional and technical needs set out.

In this section, two types of requirement are distinguished: functional requirements, which describe the capabilities and behaviors that the system must offer; and non-functional requirements, which specify constraints related to how the system should function, i.e., performance, compatibility, extensibility, and software quality.

3.1 Functional Requirements

- **Motion capture:** The system must allow the recording of the position and orientation of the HMD and hands. The position of the hands can be defined either by the use of hand tracking or by the pose of the controllers if they are being used. In addition, when hand tracking is being used, the position of the hands will be composed of the

pose of all joints defined by OpenXR.

- **Multi-platform engine support:** The system must be able to integrate with the two main application engines mentioned before, Unity and Unreal Engine.
- **Flexible configuration:** The system must offer the functionality needed to configure the information about the database to which the information is to be sent via HTTP, and to configure both the sampling frequency and the different flags that determine what data must be recorded and sent.
- **Data storage:** The system must record the data in a file locally on the HMD. This file will be in C3D format and in accordance with the specified model.
- **Data transmission:** The system must allow the remote transmission of data via a REST API to a URL defined in the configuration file.
- **Capture sessions:** The system must allow capture sessions to be uniquely identified, associating metadata with each session for subsequent analysis of this data.

3.2 Non-functional Requirements

- **Performance:** The system must be capable of capturing data in real time without significantly interfering with the main thread of the host application, minimising the impact on its overall performance.
- **Accuracy and temporal consistency:** The system must ensure temporal consistency between recorded samples, maintaining a correct relationship between capture times and stored data.
- **Portability:** The system must be compatible with different models of HMD devices based on similar virtual reality technologies, favouring its reuse in future developments.
- **Usability for developers:** The integration of the system into existing applications must be simple, without requiring major modifications to the architecture of the host project.
- **Extensibility:** The system architecture must allow for the future addition of new types of data, devices or storage formats without the need for significant changes to the core of the system.
- **Validation:** It must be possible to verify and reproduce the recorded data using external analysis tools.

Design and Implementation

This chapter presents a design that meets the functional and non-functional requirements defined in the previous chapter, considering aspects such as modularity, extensibility, portability between game engines, and accuracy in the capture and storage of movement data. The main obstacles that have been encountered while meeting these requirements have been detailed. To this end, both high-level architectural decisions and some relevant implementation details are analyzed.

First, an overview of the system and data flow is presented, describing the interaction between the different components of the library and their integration with applications developed in Unity and Unreal Engine. This chapter specifically addresses the data model used and the treatment of coordinate systems, as well as the particularities of the data capture process in each game engine, as well as the process of compiling, packaging, and integrating the library.

From a conceptual point of view, the system is based on a modular architecture in which the responsibilities are clearly separated. The logic behind structuring and packaging data is implemented in a telemetry library independent of the engine, while interaction with the game engines and the underlying operating system is carried out through specific integration layers. This modular approach facilitates both maintenance within a constantly mutating ecosystem and adaptation to possible changes within the LibreMotion project. Thus, the library has been designed with the aim of providing a flexible and reusable system

for capturing motion data in virtual reality devices, capable of being easily integrated into different game engines and adapted to future changes of the encompassing project.

The design has been geared towards fulfilling several key principles. Firstly, independence from the game engine, so that the same library can be used in both Unity-based and Unreal Engine-based applications. Secondly, system configurability, allowing relevant capture and storage parameters to be modified without the need to recompile the code. Third, consistency and accuracy in the processing of motion data, paying special attention to the data model and data capture timestamps. The system has also been designed to support different storage mechanisms, including both local data recording in a standard format and remote information transmission via web services. The design thus provides a solid and flexible foundation on which to build motion capture applications with different usage and analysis requirements.

Before continuing with a more in-depth analysis of the library's development, it is worth mentioning that this project is firmly committed to the ethics of open-source software, promoting transparency, collaboration, and community-driven improvement. Consequently, all the code developed, along with instructions or guidelines for its correct use and modification, is published on GitHub under a permissive MIT license at the following address: https://github.com/migueldulu/LibreMotion_capture.git. This license explicitly grants users the freedom to use, study, modify, and distribute the software, ensuring it remains accessible and reusable.

Due to space constraints and to preserve readability, detailed source code listings are omitted from this document. The complete code and implementation details are available in the public GitHub repository referenced above.

4.1 System Architecture

The system architecture defines the general organization of the library and the integration modules that connect it to the different game engines.

At a high level, the system architecture is organized into two main blocks: an engine-specific **integration layer** and a shared **telemetry library**. The **integration layer** is responsible for interfacing with the host application and the XR runtime, acquiring pose and input data from the underlying engine (either Unity or Unreal). The **telemetry library**, implemented as a native and engine-agnostic module, encapsulates all logic related to data processing, structuring, storage, and transmission.

Figure 4.1 presents an overview of the architecture of the system and the interactions among its main components. The library starts when the host application is launched, at which point the LibreMotion Capture runtime is automatically initialized.

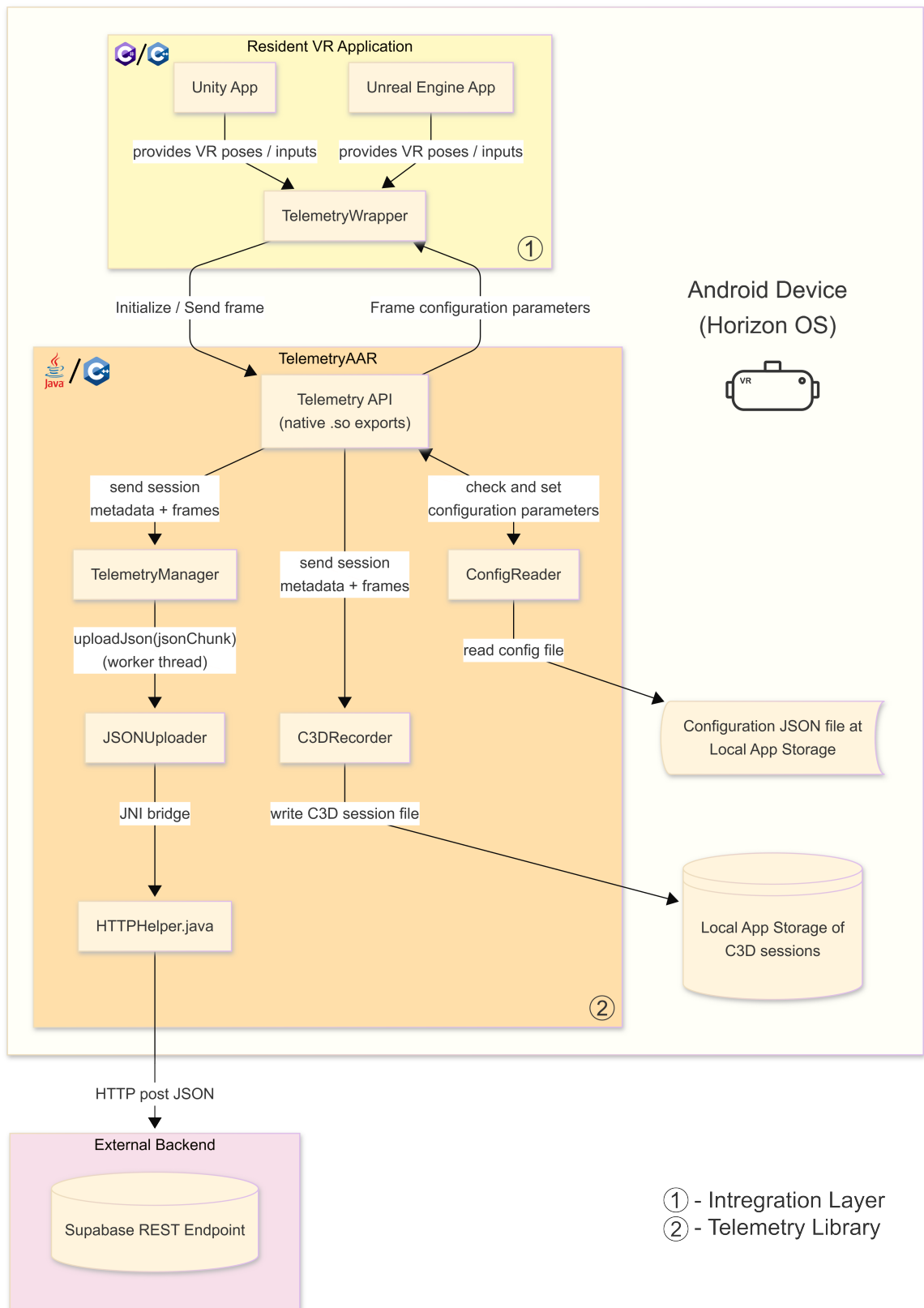


Figure 4.1: General architecture diagram

4.2 Integration Layer

Although the specific implementations for Unity and Unreal Engine differ, the overall workflow remains identical. That is why both engine implementations are abstracted through the *TelemetryWrapper*.

This *TelemetryWrapper* contains engine-specific components that connect the host application with the native **telemetry library**. Its primary responsibility is to acquire tracking data from the XR runtime through the facilities provided by each engine and forward this information to the **telemetry library** (packaged as an AAR) in a unified format. The *TelemetryWrapper* is also responsible for issuing the initialization request to the **telemetry library**, and retrieving the settings.

This way, all communication is handled through a unified *Telemetry API* that acts as a single entry point to the **telemetry library** and exposes the methods required by the host application's *TelemetryWrapper*. After its initialization, *Telemetry API* parses a configuration file and returns the found settings to the *TelemetryWrapper*. Those settings inform the way the data are captured. Then, the capture process proper begins, with the data for each frame processed and stored through two alternative mechanisms: either written locally to a C3D file within the host application's storage directory or continuously transmitted to an external database via periodic REST requests.

4.2.1 Data Capture

Corresponding source directories:

- Unreal_Implementation
- Unity_Implementation

The data capture subsystem is responsible for extracting spatial and kinematic information from the XR runtime at each frame. Due to fundamental differences in engine architecture and execution models, this functionality is realized through two distinct implementations:

- A Unity C# script, which must be attached to an empty `GameObject`.
- A custom Unreal Engine C++ plugin, attached to `VRPawn`, which integrates directly into the Unreal Actor and Component lifecycle.

Both components implement engine-specific capture logic while exposing a common data acquisition interface toward the native **telemetry library**.

This separation allows both integrations to evolve independently while guaranteeing that the resulting telemetry frames share the same internal representation and downstream behavior. Since each engine exposes different execution models and XR frameworks, certain design decisions necessarily differ between implementations.

A critical design decision in the data capture layer concerns when pose samples are taken during the frame execution. In VR systems, pose data are continuously updated by the runtime, and different engine callbacks may expose slightly different states of the tracking system.

In Unity, there are several candidate callbacks (e.g., `Update`, `FixedUpdate`, `LateUpdate`, and `OnBeforeRender`). Based on the experimental evaluation presented in Chapter 5, pose capture is performed using the `OnBeforeRender` callback, which provides the best performance and data consistency.

In Unreal Engine, pose acquisition is integrated into the component tick system, which provides detailed control over execution order and frequency. Unity exposes a point explicitly close to rendering (`OnBeforeRender`), while Unreal allows to reach the end of the game thread tick (`OnEndFrameTick`), which is the latest point consistently accessible from the user code. Although they do not correspond to the exact same execution point due to engine-specific pipelines, both callbacks represent a conceptually equivalent late-stage sampling point within the frame.

Across both engines, explicit initialization and shutdown hooks are used to guarantee that capture starts only after the XR system is fully initialized and that pending data is flushed when the application is paused or terminated.

Despite their different implementations, both capture layers follow the same conceptual pipeline:

1. Query XR tracking data from the engine (HMD, controllers, and hands).
2. Transform poses into a consistent coordinate system and unit scale.
3. Associate each sample with a precise timestamp relative to session start.
4. Forward the structured frame to the native telemetry library.

4.3 Telemetry Library

This module is responsible for managing, structuring, and storing the captured movement data. Its design is geared towards providing a clear and stable interface to the game engines, while remaining independent of the particularities of each execution environment.

To achieve these tasks, this library has been organized into a set of well-defined modules, each with a specific responsibility. This modular organization facilitates code reuse, improves understanding and maintenance of the system, and allows new functionalities to be incorporated without affecting the behavior of the other components.

4.3.1 Telemetry API

Corresponding source files:

- `TelemetryAPI.h/TelemetryAPI.cpp`

The *Telemetry API* defines the public interface used by applications, and manages the fundamental operations of the recording system lifecycle. Table 4.1 summarizes the public methods exposed by the *Telemetry API* and their role within the lifecycle of the host application, from initialization to data capture and finalization.

Table 4.1: Public methods exposed by the telemetry API and their role within the application lifecycle.

Method	Description
<code>Set_java_context</code>	On application startup, registers the Android Java context required for managing platform-dependent features such as network communication and file access.
<code>Initialize</code>	On application startup, initializes the telemetry system, loads the configuration file, and prepares all internal data structures and output mechanisms.
<code>Get_feature_flags</code>	After initialization, returns the set of enabled telemetry features based on the configuration file, allowing the host application to adapt its data capture behavior accordingly.
<code>record_frame</code>	On each frame, captures pose and input data from the XR device and forwards them to the telemetry pipeline according to the configured recording and transmission policies.
<code>Shutdown</code>	On application shutdown, finalizes the telemetry session by flushing pending data, closing files, and releasing allocated resources.
<code>force_upload</code>	On application pause or unexpected termination, forces the immediate transmission of any buffered telemetry data to the external backend to ensure data consistency.

4.3.2 Custom Data Models

Corresponding source files:

- TelemetryTypes.h
- VRTypes.h

To ensure consistent processing of motion data, the library uses a set of self-defined data models that describe how the information must be received per frame, including the information captured from the HMD, controllers, and the user's hands in a structured manner; each of these data structures groups the relevant position, orientation, and status information together. These models are designed as a superset of the two standards the library must interface with (OpenXR and C3D), ensuring that no relevant information is lost during the abstraction process.

This approach abstracts the differences between game engines and provides a common representation that is independent of the data source, facilitating both internal data processing and subsequent serialization into different output formats. Each received frame follows the following structure:

```
struct VRFrameDataPlain {  
    double timestampSec; //real time when frame was captured  
    VRPosePlain hmdPose; //HMD pose  
    ControllerStatePlain leftCtrl; //left controller  
    ControllerStatePlain rightCtrl; //right controller  
    JointSamplePlain leftHandJoints[30];  
    int leftHandJointCount;  
    JointSamplePlain rightHandJoints[30]; //right hand joints  
    int rightHandJointCount;  
};
```

Hand joints have their own class because, in addition to the position and rotation data, each has a different id corresponding to the OpenXR standard. This standard establishes 26 joints per hand, but the array is set to 30 positions to avoid any possible implementation problems in Unreal Engine. This quirk will be detailed later.

VRPosePlain is a simple class composed of two one-dimensional float arrays with 3 and 4 cells, representing position and rotation respectively.

Lastly, the controller structure is a little more complex than the HMD's because controllers info includes buttons, joysticks and triggers.

```
struct ControllerStatePlain {
    VRPosePlain pose; //pose in tracking space
    unsigned int buttons; //bitmask corresponding to configuration flags
    float trigger; //analog trigger value from 0 to 1
    float grip; //analog grip value from 0 to 1
    // Analog joystick position
    float stickX;
    float stickY;
    int isActive; //0 if controller is not tracked that frame
};
```

In addition, two more structures are defined that are not movement-related, but contain metadata. The first establishes the metadata received each session from the engine (session ID and device identification), as the example provided below:

```
{
    "session_id": "HostAppName-20251128-randomCode",
    "device_info": "DeviceName - AppName"
}
```

Another second structure defines all the configuration settings used internally by the library, mainly for JSON uploading. Regarding the database connection, there are three important fields: the database URL, the database security key (if needed), and frames per file sent. Session id, device information, and configuration flags are used for JSON writing.

```
{
    "endpoint": "https://theDataBaseUrl/table1",
    "api_key": "<anon_key>",
    "frames_per_file": 300,
    //configuration flags
    "hand_tracking": true
    ...
}
```

4.3.3 Configuration Management

Corresponding source files:

- `ConfigReader.h/ConfigReader.cpp`

Table 4.2: Main functions in `configReader` and their role within the application lifecycle

Function	Description
<code>getExpected-ConfigPath</code>	Resolves the expected absolute path of <code>initialConfig.json</code> by deriving the package name (process name) and building the standard Android app-specific storage route.
<code>readFileToString</code>	Loads the full contents of a file into memory as a string, enabling subsequent JSON parsing and validation of configuration data.
<code>setConfig</code>	Parses <code>initialConfig.json</code> (if present) and fills <code>UploaderConfig</code> by applying defaults and overriding only the keys found in the JSON file.
<code>getFrameRate</code>	After initialization returns, returns the <code>frameRate</code> field from <code>initialConfig.json</code> (or a default if missing), so the engine can lock its sampling policy to the configured value.
<code>getFeature-FlagsBitmask</code>	After initialization returns, converts the enabled feature flags in <code>UploaderConfig</code> into a compact bitmask so the host application can quickly adapt which fields it captures and sends.

The library incorporates a configuration management feature (*ConfigReader*) based on an external JSON file stored within the HMD itself, at the host app files path. This allows key system parameters to be defined without the need to recompile the code. All configurable parameters are listed in Table 4.3.

This aspect was a key design consideration, as each medical study requires a different set of parameters depending on its specific objectives. In addition, it is essential to allow the explicit definition of the location of the database URL for the JSON module to work properly.

This module locates the configuration JSON file based on the host application's package name. Once the file is found, the provided values are validated and default values are applied

Parameter	Type	Meaning
Endpoint	String	Database URL where the JSON should be sent
ApiKey	String	Security key for HTTP calls to the database (if needed)
FramesPerFile	Integer	Frames accumulated before sending a JSON payload
FrameRate	Integer	Maximum framerate recorded
HandTracking	Boolean	True if hands should be recorded
PrimaryButton	Boolean	True if controllers' primary button should be recorded
SecondaryButton	Boolean	True if controllers' secondary button should be recorded
Grip	Boolean	True if controllers' grip should be recorded
Trigger	Boolean	True if controllers' trigger should be recorded
Joystick	Boolean	True if controllers' joystick should be recorded

Table 4.3: Configurable recording parameters

whenever necessary, ensuring proper behavior even in the presence of incomplete or incorrect configuration parameters.

From a design perspective, JSON was chosen as the configuration file format because it is easy to edit with any text tool and simple to comprehend. The idea of having a variable configuration provides great flexibility and facilitates the adaptation of the system to different usage scenarios. The configuration is read during the initialization phase, and its values are propagated to the different modules of the library in a controlled manner.

4.3.4 C3D Recorder

Corresponding source files:

- `C3DRecorder.h/C3DRecorder.cpp`

For local storage of motion data, the library includes a module (*C3DRecorder*) responsible for generating files in C3D format. This module relies on the `ezc3d` library [20], which provides a battle-tested implementation of the C3D standard. C3D files are written in a specific binary format, and trying to develop from scratch a whole C3D file writer would have been far too complex considering this is not the main scope of this work. `ezc3d` provides all formatting and writing logic and allows the C3D Recorder module to generate the necessary points, labels, and analog channels, organized into frames with an explicit sampling frequency. The C3D files are stored in the same path file as the configuration JSON, and each file will be named after a unique `sessionId` for easier identification.

The data model and coordinate system processing are the most critical aspects of the developed system. Incorrect or inconsistent representation of spatial information would

Table 4.4: Public methods in C3DRecorder and their role within the application lifecycle

Method	Description
C3Dinitialize	Initializes the C3D recording session by preparing output paths, selecting metadata, and storing the target sampling frequency for the file.
C3DrecordFrame	On each frame, appends the incoming VR pose and input sample to the in-memory buffer that will later be exported to a single C3D file.
C3Dflush	Forces a partial flush/synchronization step without finalizing the session.
C3Dfinalize	On application shutdown, finalizes the recording session and triggers the generation of the final .c3d file from the accumulated frames.
C3DisInitialized	Allows the caller to verify whether the recorder is ready before attempting to record or finalize.
writeC3D	On application shutdown, performs the actual C3D file generation: builds channel layouts and serializes all buffered frames into the final binary file.

compromise the validity of the captured data and limit its subsequent use in biomechanical analysis or artificial intelligence model training tasks. For this reason, the system design pays special attention to both the structure of the stored data and the definition and transformation of the reference systems used throughout the capture process.

The captured motion data are organized into discrete frames, with each frame represents the user’s full kinematic state at the specific capture time. This includes the pose of the HMD, controllers, and, when active, hand joints. However, as discussed in Chapter 2, the nature of motion data obtained from XR devices differs substantially from that produced by traditional optical motion capture systems based on multiple cameras and physical markers. This discrepancy gives rise to several fundamental challenges that must be addressed when adapting XR-derived data to conventional biomechanical formats such as C3D.

4.3.4.1 Representation of Orientation Data and Timestamp in C3D

The C3D format was originally designed to represent biomechanical models using body-mounted markers, where each marker is defined solely by its three-dimensional position. In contrast, the motion data captured in this work include not only positional information

but also full orientation data for each tracked element, which is particularly relevant in the context of XR-based motion analysis.

However, the C3D standard does not natively support the association of orientation information with individual 3D points. This limitation prevents a direct and explicit representation of quaternion-based poses within the core marker data structure, requiring alternative mechanisms to store rotational information without compromising compatibility with existing C3D analysis tools.

Also, traditional optical motion capture systems typically operate at very high and stable sampling frequencies under controlled laboratory conditions. As a consequence, C3D files do not store an explicit timestamp for each recorded frame; instead, temporal information is calculated directly from a single global sampling frequency defined in the file header. Under these conditions, temporal error is assumed to be negligible.

In contrast, motion data acquired from HMDs are subject to greater temporal variability. The effective sampling rate depends on factors such as rendering load, system scheduling, and computational constraints of the device. This results in small but non-negligible variations in the time interval between consecutive samples, often on the order of milliseconds. When such data are represented using a strictly fixed sampling frequency, temporal inaccuracies may be introduced, potentially affecting time-sensitive biomechanical analyses.

To address both limitations, analog channels available in the C3D format have been used. These channels allow additional information to be stored alongside marker data; usually this information compounds physiological data (electromyography, blood oxygen saturation, etc.) or physical magnitudes (force, pressure, etc.). Here they have been used to record orientation quaternions of all captured points, supplementary controller data, and the precise timestamp at which each sample is recorded. This explicit timestamp can be particularly relevant in scenarios that require synchronization across multiple systems.

4.3.4.2 Additional Points, Coordinate Axes and Measurement Units

In order to adapt the data format to the specific needs of the LibreMotion project, two additional features have been added to the standard model. These extra features are customized tags for gait analysis and the capture of kinematic information from virtual reality devices, as well as changes to the coordinate system. They are needed to provide compatibility with existing biomechanical analysis tools by matching the Plug-in-Gait model, which is frequently used for traditional C3D visualization.

The Plug-in Gait model details the recommended location of the human body position markers. The points obtained from the HMD software, such as the center point between the eyes and the center point of the wrist, do not completely correspond to any point of the

Plug-in Gait model, so additional points have been created that calculated. These points match the markers located on both sides of the forehead and on both sides of each wrist in the Plug-in Gait model. The representation in figures 4.2 and 4.3 shows in red the synthetic points that have been calculated from the original captured points (in green).

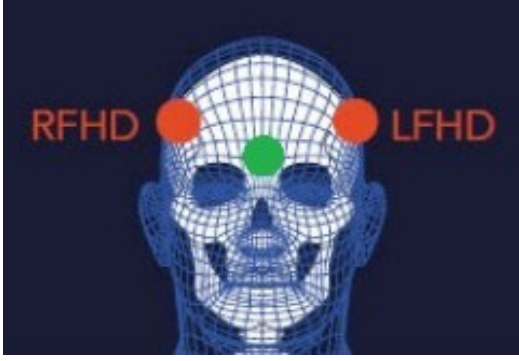


Figure 4.2: Head markers

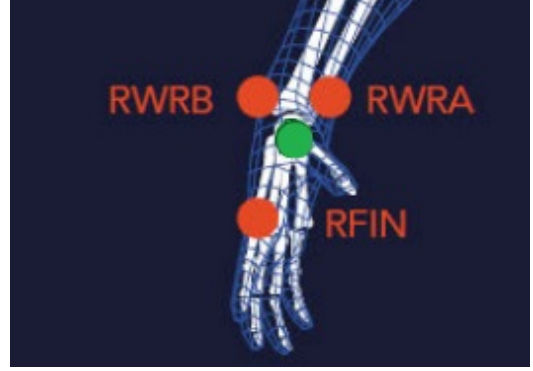


Figure 4.3: Wrist markers

To determine the position of the synthetic points, the average width of the human head and wrist were approximated [21] and used to put these new points to the left and to the right (frontal plane) of the measures points. These calculations were not a direct sum of distances, due to the 3D coordinate system. First, the vector equivalent to the width of the head or wrist must be rotated to match the orientation of the tracked point:

$$\vec{v} = (Calculated_Width, 0, 0)$$

To rotate a vector $\mathbf{v}$ by a unitary quaternion q , the following formula is used:

$$\mathbf{v}_{rotated} = q\mathbf{v}q^*$$

where q^* is the conjugate of the quaternion q .

Unlike OpenXR, C3D does not define a specific coordinate system, so using the same as OpenXR should be acceptable. However, after examining several visualizations of various C3D files, we concluded that all visualization software tools use a right-handed coordinate system in which the ground is represented by the X-Y plane. This is specially important, since OpenXR uses a left-handed coordinate system. Therefore, the data were converted by performing a simple substitution:

$$X_{new} = -Z_{old}, \quad Y_{new} = -X_{old}, \quad Z_{new} = Y_{old}$$

For quaternions, a fixed rotation must be applied to the input quaternion, using what is known as the Hamilton product [22].

Another problem is that, even though C3D indicates that the measurement units must be specified in the header of the C3D file, many visualization tools ignore this fact and assume that the units are millimeters. So, to avoid problems with any visualization software, we decided to use this measurement unit (and properly fill the header).

As described in Chapter 2, there are two possible reference spaces. For this work the tracking space was selected as reference system. This choice allows a consistent behavior of the HMD and controllers, independent of the application used or patient environment.

Finally, although only a subset of the captured points can be directly mapped to the Plug-in Gait marker model, the original tracked points provided by the XR system are preserved in the data representation, so any software conscious of their existence can make full use of them. For this, a systematic naming convention consistent with the Plug-in Gait terminology was defined for all supplementary points, including the full set of hand joints exposed by OpenXR. This approach ensures that both traditional biomechanical markers and extended XR-specific points remain available for downstream medical analysis, without restricting the dataset to a single reference model.

4.3.5 JSON Uploader

Corresponding source files:

- `TelemetryManager.h/TelemetryManager.cpp`
- `JSONuploader.h/JSONuploader.cpp`
- `HttpHelper.java`

In addition to local recording in C3D, the library provides an optional mechanism to transmit captured frames to an external service through HTTP requests. This functionality enables the construction of remote datasets in a centralized database, facilitating subsequent extraction, visualization, and large-scale analysis. The JSON uploader is an output module that consumes the engine-agnostic internal data model and serializes it into JSON payloads without requiring any engine-specific logic. This is implemented through three components: *TelemetryManager*, *JSONUploader* and *HTTPHelper*.

This module receives frame samples through the core recording pipeline and stores them in memory until a configurable threshold is reached (*FramesPerFile*, Table 4.3). Once this number of frames is collected, the module generates a JSON document containing both session metadata (`session_id` and `device_info`) and an array of frames, each including timestamped poses for the HMD and controllers and, when enabled, hand joint samples. This batching approach reduces network overhead by avoiding per-frame HTTP requests,

Table 4.5: Public methods in JSONUploader and their role within the application lifecycle

Method	Description
<code>setJavaContext</code>	Stores the <code>JavaVM*</code> and a global <code>Activity</code> reference required to access Java Native Interface (JNI), resolve classes from the AAR, and perform platform-dependent operations.
<code>initialize</code>	Initializes the JNI bridge needed for HTTP requests.
<code>uploadJson</code>	Sends a JSON payload to the configured backend by delegating the HTTP request to the Java-side helper through JNI.
<code>shutdown</code>	Releases stored JNI references and shuts down any uploader-related state to avoid resource leaks.
<code>callJavaMakeRequest</code>	During runtime (internal helper), performs the low-level JNI call that builds the Java request (method, URL, body, headers), attaches/detaches the current thread if needed, and executes the Java HTTP helper.

improves throughput under mobile networking conditions, and provides a natural compromise between latency and reliability.

Although the core library is implemented in C++ for portability across engines, the HTTP upload layer is delegated to a small Java helper component (*HTTPHelper*). This is because on Android-based HMDs stable networking is more naturally integrated with the Java/Kotlin runtime and its platform APIs (TLS handling, connection management, and lifecycle-aware behavior). Implementing the upload logic in Java reduces complexity in the native layer and avoids adding extra third-party dependencies to the NDK build, while still allowing the C++ core to remain engine-independent.

A key requirement in VR applications is graceful termination and safe resource handling. For this reason, the upload module is designed to support explicit finalization through the public API (e.g., forced upload on shutdown) and to tolerate unexpected interruptions. When the host application closes, the library attempts to flush pending buffered frames and trigger an upload request so that partially recorded sessions are not lost.

All network parameters are externalized in the configuration JSON described previously, including the endpoint URL and optional API key. Not hardcoding the credentials is a basic approach to reduce the risk of accidental exposure in public repositories. Also, the credentials can be rotated without rebuilding the library binaries.

To make the uploader usable from both Unity and Unreal Engine on Android-based HMDs, the Java helper and the native `.so` library are packaged together into a single

Table 4.6: Public methods in `TelemetryManager` and their role within the application lifecycle

Method	Description
<code>initialize</code>	Prepares the telemetry pipeline by configuring buffers, associating the <code>JSONUploader</code> , and starting any background worker needed for chunked transmission.
<code>recordFrame</code>	Each frame stores incoming samples into the telemetry buffer and applies the configured number of frames per chunk).
<code>flushAndUpload</code>	On application pause or unexpected termination, forces immediate serialization and transmission of any buffered chunks to reduce the risk of data loss.
<code>shutdown</code>	On application shutdown, stops background processing, flushes remaining data, and releases resources associated with the telemetry transmission pipeline.
<code>serializeAndSend</code>	During runtime, converts a chunk of frames into the JSON payload expected by the backend and delegates the network request to <code>JSONUploader</code> .
<code>workerLoop</code>	During runtime, implements the background thread loop that waits for pending chunks and uploads them asynchronously to avoid blocking the render thread.

AAR. This packaging choice provides two practical advantages: the engine-side integration becomes simpler, as the developer imports a single artifact that already contains both native and Java layers; and dependency management is centralized, ensuring that the correct Java class and native binaries are deployed together. As a result, the engines interact only with the unified C++ entry point (*Telemetry API*), while the AAR encapsulates all Android-specific communication details required by the JSON uploader.

Table 4.7: Main methods in `HTTPHelper` and their role within the telemetry upload lifecycle

Method	Description
<code>makeRequest</code>	Performs the HTTP request used by the native uploader (via JNI). If invoked on the Android main thread, it automatically offloads the request to a worker thread to avoid <code>NetworkOnMainThreadException</code> ; otherwise it executes the request directly.
<code>runInWorker</code>	Executes <code>doRequest</code> on a single-thread executor and waits up to 60 seconds for completion, ensuring the UI thread is not blocked by network I/O.
<code>doRequest</code>	Implements the actual HTTP transaction using <code>HttpURLConnection</code> : applies headers, writes the request body, reads the response bytes, and if the status code is not 2xx appends a trailing <code>HTTP_STATUS:<code></code> marker to the returned payload.

5.1 Detection and Correction of Data Loss During Capture

To begin the validation process, a smoke test was performed. It confirmed that the recorded pose values (HMD and hands/controllers) were consistent, after applying the initial corrections to the coordinate system and measurement units, with the expected magnitudes in the real-world (for example, HMD height matched the approximate height of the participant).

However, when plotting the same data as time series and trajectories, an unexpected artifact became evident: periodic gaps where no samples were recorded. In the plotted trajectories, these gaps appeared as discontinuities (sudden “jumps” from one position to a later position), indicating that intermediate frames were missing rather than the movement itself being discontinuous. This can be seen in Figure 5.1, which shows a low frequency recording of a circular movement using a controller, with the red markers representing the periodical frame loss. In this case a low sampling frequency and a slow movement were used for an easier data understanding, but with higher parameters the frame loss becomes more evident.

To diagnose this issue, several capture sessions were recorded while varying the telemetry configuration parameters. The most relevant factor was the batch size used for network transmission (i.e., of the configurable parameters *FramesPerFile* in Table 4.3). The periodicity of the gaps closely matched the transmission schedule: when the system was configured

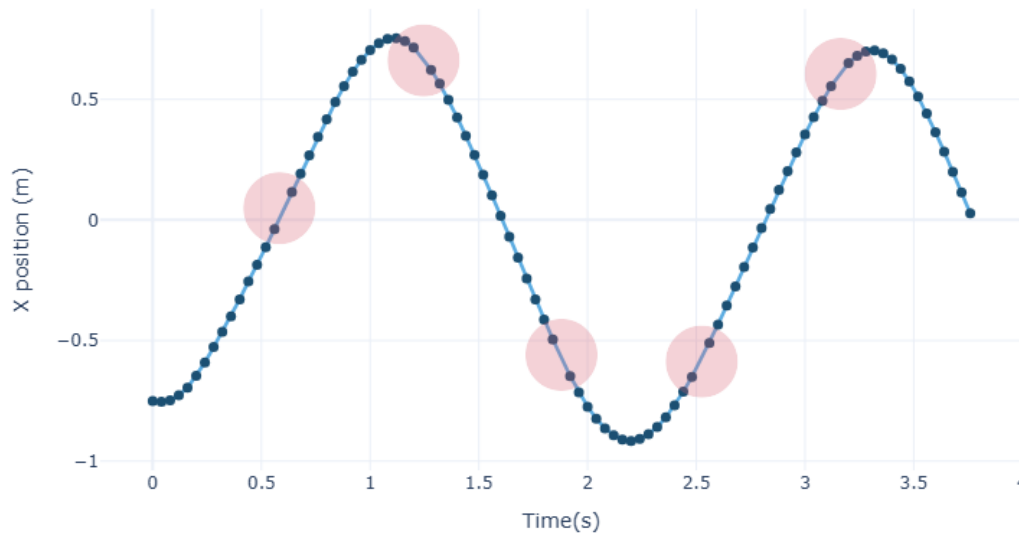


Figure 5.1: Missing frames on simple movement

to send more frequently (smaller batches), the gaps became shorter and more frequent; conversely, larger batches produced longer discontinuities. This strong correlation suggested that the capture loop was being interrupted during the upload step.

The root cause of the problem was related to the fact that data sampling and sending are executed on the same thread. Each time the *framesPerFile* threshold was reached, the system had to first serialize the block of frames to JSON and then make an HTTP request to the backend. These operations have a significant time cost (especially on Android devices), and their duration depends on factors such as JSON size, network latency, and processor performance. If these tasks are performed on the thread responsible for capture, during that interval, the system stops executing *recordFrame* at the target frequency, resulting in effective sample loss. In the recorded signal, this translates into sections without data, visible as discontinuities in the trajectory.

To eliminate this effect; an initial attempt was forcing the capture loop to continue sampling frames while the upload operation was still in progress. However, this approach led to severe degradation of the overall system performance. In practice, the increased CPU load caused by concurrent serialization, network transmission, and high-frequency sampling resulted in a noticeable drop in HMD rendering performance, negatively affecting the stability and usability of the VR application. Due to these limitations, this solution was discarded, motivating the adoption of a more complex but robust architectural redesign based on asynchronous processing; the data flow was redesigned by explicitly decoupling real-time capture from the serialization and sending process.

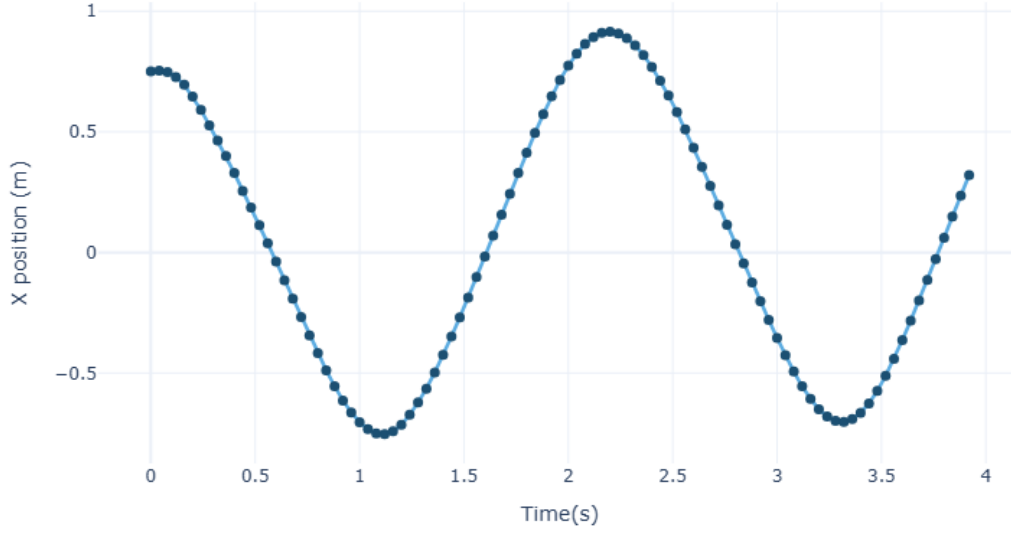


Figure 5.2: Similar movement with optimized processing

The final solution solves this issue by:

1. Maintaining a frame buffer in the capture thread.
2. When *framesPerFile* is reached, converting the buffer into a chunk using a swap operation and immediately releasing the synchronization lock associated with the capture.
3. Queueing that chunk in a queue protected by an independent *mutex* and notifying a worker thread.
4. Processing serialization and HTTP sending in a worker thread, so that the time cost of sending does not interrupt data collection.

This change transforms the system into a producer–consumer model: the main thread acts as a high-frequency sample producer, while the worker thread consumes blocks and uploads them asynchronously. Additionally, during shutdown, a flush of the pending buffer is forced (*flushAndUpload*), and the worker is waited for to finish (*join*), ensuring that the latest data is also transmitted.

After applying this modification, the recorded trajectories no longer show periodic gaps and display temporal continuity in line with the target sampling frequency. Therefore, it was confirmed that the problem observed was not inherent to the device’s tracking, but a direct consequence of a blockage caused by serialization and sending on the same capture thread. The Figure 5.2 shows a similar test with a circular movement, but with the corrected process, the frame loss has disappeared.

5.2 Temporal Stability Analysis and Sampling Strategy

One of the main objectives of this work was to ensure the highest possible temporal stability in motion data capture. This aspect is particularly relevant because many traditional biomechanical analysis formats and tools assume a strictly constant sampling frequency, whereas modern virtual reality systems introduce greater temporal variability associated with real-time rendering, motion prediction, and user tracking.

In order to identify the most appropriate time to request the device's pose, an experimental analysis was carried out in Unity, evaluating different points in the application's life cycle. Specifically, the capture of the HMD position was compared in the `Update`, `LateUpdate`, `FixedUpdate` and `OnBeforeRender` methods, recording both the resulting spatial trajectories and the temporal stability of the samples obtained.

To isolate the effect of the engine and the XR system, the tests were performed in a minimal scene and run directly on the device (standalone), avoiding the use of streaming tools such as Meta Link. In addition, the refresh rate of the viewer and the engine was explicitly forced to achieve rates close to 120 Hz.

This way, it was evaluated whether the moment of pose capture had a significant influence on the shape of the recorded trajectory. To do this, two experiments were designed:

1. Circular translation movement of the controller at approximately constant speed.
2. Same movement with variable speed, introducing deliberate accelerations and decelerations.

In both cases, the trajectories were captured using `Update`, `LateUpdate`, `FixedUpdate`, and `OnBeforeRender` and compared visually.

From a theoretical point of view, and assuming a direct discrete sampling of the underlying motion signal, temporal aliasing phenomena would be expected when the sampling frequency is not sufficiently high with respect to the dynamics of the movement, in accordance with Nyquist's theorem [23].

In the experiment involving rotation at approximately constant speed, this temporal aliasing would manifest itself in the form of unevenly spaced points along the circular trajectory, even when the actual speed of the movement was uniform. The circumference reconstructed from the samples would present irregularities in the density of points, evidencing a discrepancy between the actual movement and its sampled representation.

Similarly, in the variable-speed rotation experiment, it would be expected to observe small discontinuities or local irregularities in the trajectory, such as peaks, sudden changes in slope, or step-like effects, in a curve that should ideally be smooth.

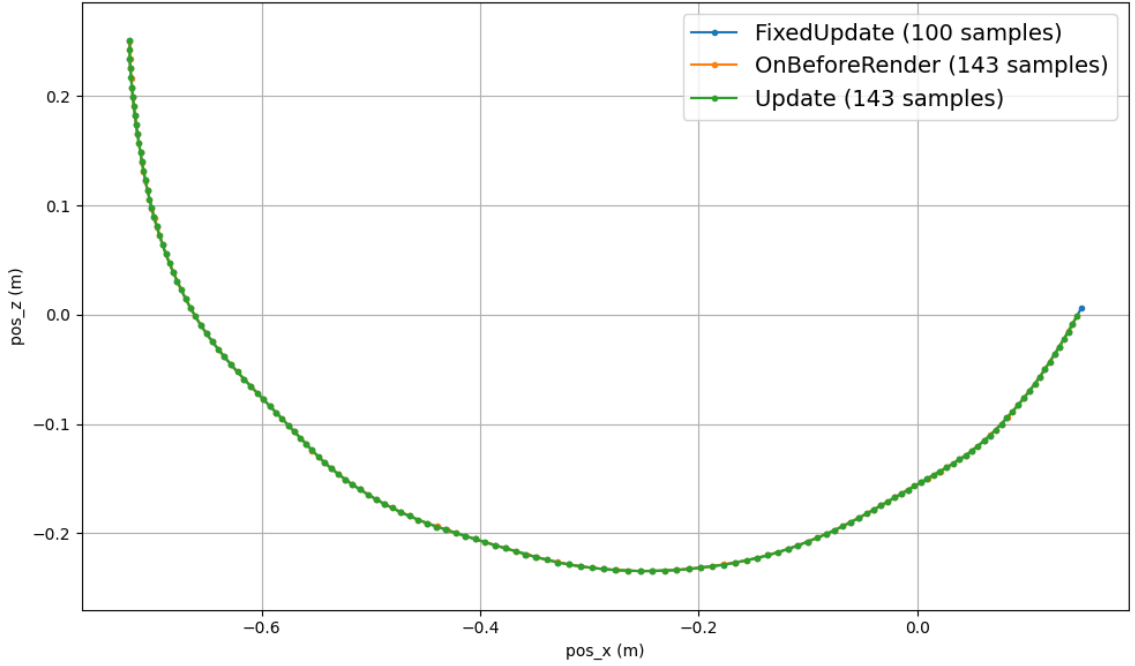


Figure 5.3: X-Z circular trajectory - 2 sec interval

However, in both experiments carried out, no temporal aliasing patterns were observed in the distribution of points, nor were there any noticeable spatial discontinuities, even during movements with deliberate accelerations and decelerations.

The obtained trajectories maintained a continuous and smooth evolution in both cases and were accurately similar, indicating that the pose delivered by the XR runtime is the result of a prediction-corrected estimate process optimized for rendering latency; and not an instantaneous reading of the raw values.

Figure 5.3 represents the evolution of a single positional axis as a function of time. The absence of step-like artifacts or discontinuities supports the hypothesis that the pose provided by the XR runtime is temporally interpolated. It should be noted that the trajectories obtained with the three evaluated methods are nearly identical and therefore overlap almost perfectly in the plot, appearing as a single curve.

Since spatial analysis did not reveal significant differences between methods, the decisive criterion for selecting the optimal capture point became the temporal stability of the samples.

To this end, jitter, defined as the variation in the time interval between consecutive samples ($\sigma(\Delta t)$), was analyzed for each of the methods evaluated. Figure 5.4 shows the jitter values achieved by the different methods. Note that the larger spikes correspond to critical events, such as the app going pause mode or shutting down, so they can be safely ignored. In this analysis, the three methods showed clear differences:

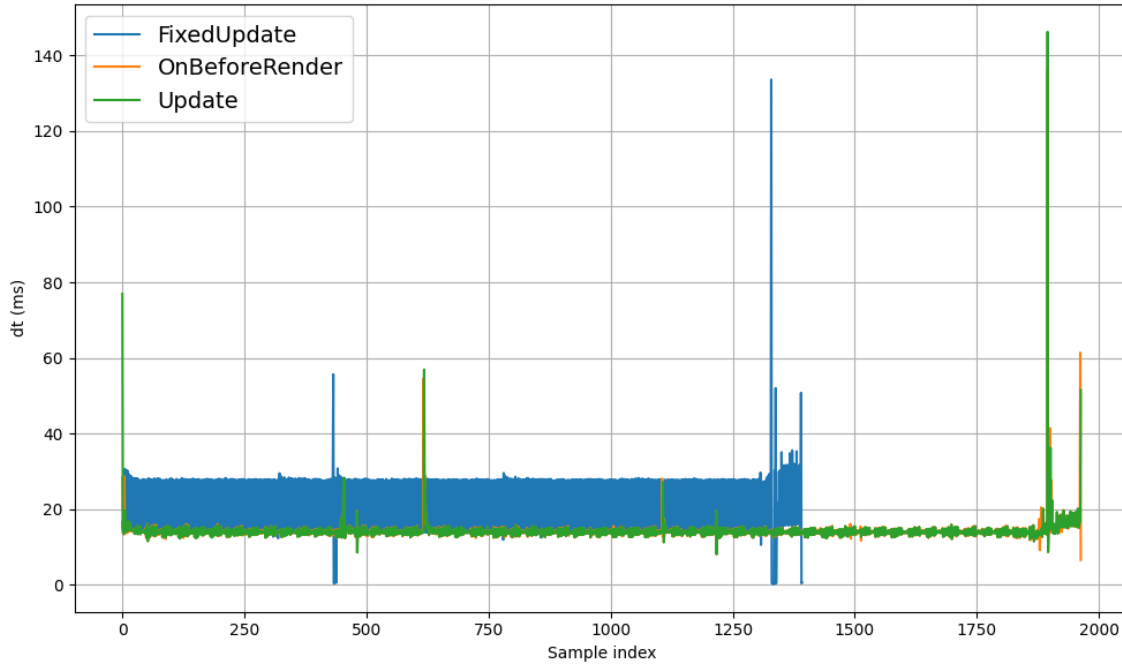


Figure 5.4: Jitter comparison across callbacks

- `FixedUpdate`, being linked to a fixed simulation timestep independent of the tracking system and the rendering pipeline, showed significantly greater temporal variability.
- `Update` and `OnBeforeRender` offered good temporal stability, as they were executed once per logical frame of the engine.

Since the results for the `Update` and `OnBeforeRender` callbacks were very similar, the jitter was computed analytically to strengthen the visual conclusions. This can be seen in Table 5.1 which confirms our findings regarding the low stability of `FixedUpdate`, and additionally shows that in both experiments, `OnBeforeRender` is marginally more reliable than `Update`.

Table 5.1: Temporal jitter analysis for different pose sampling callbacks.

Callback	Experiment	Freq. (Hz)	$\overline{\Delta t}$ (ms)	$\sigma(\Delta t)$ (ms)
FixedUpdate	Constant speed	50.03	19.99	7.85
Update	Constant speed	70.30	14.22	3.76
OnBeforeRender	Constant speed	70.43	14.20	3.41
FixedUpdate	Variable speed	49.77	20.09	7.60
Update	Variable speed	71.14	14.06	2.26
OnBeforeRender	Variable speed	71.29	14.03	1.77

The results obtained lead to two complementary conclusions. First, the similarity of the spatial trajectories, even under conditions of non-uniform motion, indicates that the pose delivered by the XR runtime is the result of a process of fusion, filtering, and temporal interpolation of the available sensors, rather than an instantaneous reading of the raw values.

Second, although this interpolation guarantees smooth and stable trajectories from a spatial point of view, the instant at which the pose is requested does have a significant influence on the temporal stability of the recorded data. Analyzing the data shown in the table it could be concluded that *OnBeforeRender*, given that it is executed immediately before sending the frame to the virtual reality composer, provides the best temporal alignment with the frame finally presented to the user and the greatest temporal consistency.

5.2.1 Specific Considerations in Unreal Engine

A separate experimental evaluation equivalent to the one performed in Unity was not required for the Unreal Engine integration. The underlying XR tracking data is provided by the same OpenXR runtime, and the **telemetry library** applies identical data handling and timestamping logic in both engines. As a result, the experimental analysis presented before is directly applicable from a conceptual point of view.

UE does not expose a callback directly equivalent to Unity’s *OnBeforeRender*. Instead, *OnEndFrameTick* was used for pose acquisition, so it is performed at the end of the game thread execution. While these two approaches do not correspond to the exact same execution point due to differences in engine pipelines, both represent a conceptually equivalent late-stage sampling point within the frame, providing access to a temporally stable pose estimate for the current frame.

In this context, using the *OnEndFrameTick* callback does not imply access to less processed sensor data, but rather capture the last valid estimate corresponding to the current frame. However, this point guarantees that the logical frame has ended and that all actors and components have been updated, providing greater temporal consistency between the different recorded elements.

In conclusion, these results justify the design decision of aligning pose capture with the latest frame-synchronized callback available in each engine, ensuring maximal temporal consistency while relying on the XR runtime to provide a stable and interpolated pose estimate.

5.3 Performance Analysis

In order to evaluate the impact of the proposed telemetry library on system performance, a preliminary resource consumption analysis was conducted. The experiment consisted of executing a series of motion sequences with moderate repeatability, similar durations, and a consistent application frame rate across all trials, using the same Unity application, either with or without the Libremotion Capture library present and enabled. To assess the worst-case scenario in terms of computational load, the most demanding library configuration was enabled during the tests, including hand tracking, JSON transmission and C3D file generation. This configuration represents the maximum expected workload during normal operation of the system.

The performance metrics analyzed were selected to provide a representative overview of both rendering stability and system resource usage. Specifically, the following indicators were evaluated: Average FPS (Frames per Second), stale frame count, CPU utilization, GPU utilization, App GPU Time, and CPU/GPU performance levels. These metrics were captured using *the OVR Metrics Tool*, a Meta-developed profiling application for performance analysis on standalone VR devices. In addition to displaying real-time information through an on-device overlay, this tool allows the recorded data to be exported as CSV (Comma Separated Value) files for offline analysis.

Figure 5.5 presents four different tests performed on the same application, labeled as *LM Capture* when the library component was present and used, and as *NonLib* when it was not included. The results show that the observed variability between independent executions of the same configuration is comparable to the differences measured between executions with the telemetry library enabled and disabled, for one test it would appear that the library version is even less demanding (which reinforces that variability dominates the result). As a result, no systematic degradation of performance can be attributed to the integration of the library.

A minor reduction in frame rate is observed during the initial milliseconds following application startup when the telemetry library is enabled, during its initialization, but it does not persist after a few frames. Beyond this initialization phase, no sustained impact on frame rate stability or resource utilization is detected.

Overall, these results support the conclusion that the proposed telemetry library introduces a limited overhead that does not compromise real-time performance under the tested conditions. While further experiments with stricter statistical control and repeated trials would be required for a complete characterization, the presented analysis provides evidence of the feasibility of the solution from a performance and resource consumption perspective.

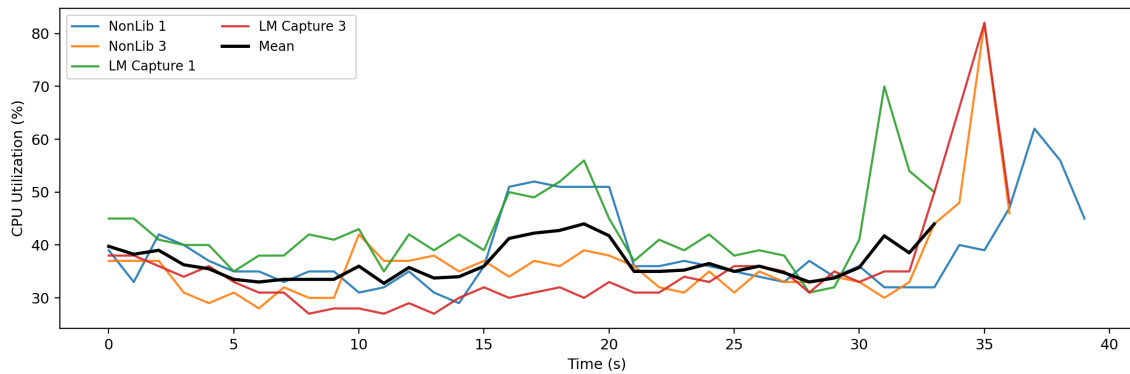


Figure 5.5: CPU utilization across 4 different test

5.4 Validation of Data Persistence Mechanisms

This section validates the two persistence mechanisms implemented in LibreMotion Capture: Real-time transmission of motion frames as JSON to a Supabase backend and local recording of sessions into C3D files. The validation is focused on demonstrating that both pipelines operate end-to-end and that the stored data remain consistent with the poses observed in the XR application.

5.4.1 Validation of JSON Transmission to Supabase

To validate network persistence, a local Supabase instance was deployed using the official Docker-based distribution provided by the Supabase project [24]. The standard distribution was used without modification, providing a controlled environment in which the database schema, REST endpoint availability, and authentication settings could be reliably reproduced.

The pipeline was validated through two complementary checks:

- **Initial configuration parsing.** The library reads an external JSON configuration (e.g., endpoint, API key, frame rate, enabled channels) at initialization time. This is the configuration json file used for this case:

```
{
  "endpoint": "http://192.168.1.45:54323/rest/v1/rpc/insert_vr_frame",
  "apiKey": "sb_publishable_6lmLqzJKoN3_AMLn--wZZg_ch1QWcaf",
  "framesPerFile": 150,
  "frameRate": 72,
  "handTracking": true,
  "primaryButton": true,
```

```

    "secondaryButton":true,
    "grip":true,
    "trigger":true,
    "joystick":true
  }

```

The endpoint field points to the Docker container hosting the Supabase backend, which is accessible through the local network to which the HMD is also connected. The remaining parameters follow the library's default configuration, enabling the capture of all available input variables and setting a sampling frequency of 72 Hz. Captured data are grouped into batches of 150 frames before being transmitted to the backend.

- **End-to-end POST delivery and server-side persistence.** During execution, the library assembles a JSON payload per batch, containing timestamps and the pose data for HMD and controllers or hands. The delivery was validated by confirming successful HTTP responses and inspecting the received rows through Supabase tooling. As seen in Figure 5.6, which illustrates an example of a payload that has been received and stored in the database, the data is being saved correctly. The following is a complete example of a single frame received:

```

[{"session_id":"Prueba2aLibreria-20260108-041033-28a58e",
"timestamp":2.24047,
"device_info":"Oculus Quest - Prueba2aLibreria",
"head_pose":{"orientation":[-0.0984737,-0.378052,-0.0700182,0.917866],
"position":[1.21381,1.32096,-0.346618]},
"controllers":{"left":{"tracked":false,
"orientation":[0,0,0,1],
"position":[0,0,0],
"trigger":0, "grip":0, "primary":false,
"secondary":false,"joystick_x":0,"joystick_y":0},
"right":{...},
"hands": {"left":{"joint_count":26,
"joints":[{"id":0, "state":35,
"orientation":[-0.378049,-0.453615,0.576065,0.565208],
"position":[1.13923,1.04989,-0.660837],
"has_pose":true},
... ,
{"id":25, ... }]},
"right":{"joint_count":26,"joints":[...]}}},

```

id	int8	session_id	text	tim...	fi...	device_info	text	head_pos...	flo...	head_pos...	flo...	head_pos...	flo...	head_rot_x	float8	head_rot_y
1		Prueba2aLibreria-202	0			Oculus Quest - Prueb		1.21381		1.32096		-0.346618		-0.0984737		-0.378052
2		Prueba2aLibreria-202	0.013916			Oculus Quest - Prueb		1.21377		1.32098		-0.346693		-0.098134		-0.377618
3		Prueba2aLibreria-202	0.027808			Oculus Quest - Prueb		1.2137		1.32095		-0.346851		-0.0979144		-0.377357
4		Prueba2aLibreria-202	0.042001			Oculus Quest - Prueb		1.21361		1.32088		-0.34697		-0.0978874		-0.377188
5		Prueba2aLibreria-202	0.055865			Oculus Quest - Prueb		1.21344		1.32077		-0.347018		-0.0987422		-0.376926
6		Prueba2aLibreria-202	0.07432			Oculus Quest - Prueb		1.21328		1.32073		-0.347016		-0.0989781		-0.377343

Figure 5.6: Screenshot from Supabase web server tool showing stored frames

The result of this validation is the following:

1. The configuration parameters effectively control the telemetry features.
2. The backend receives and persists frames without structural inconsistencies (missing fields, wrong types, or malformed arrays). In addition, basic integrity checks were applied, such as verifying monotonic timestamps and ensuring that the number of pose elements matches the enabled channels (e.g., hand joints only present when hand tracking is enabled).

5.4.2 Validation of C3D Session Recording

C3D persistence was validated with a methodology adapted to the nature of the format. Since C3D is a binary container, direct manual inspection is not practical; therefore, correctness was evaluated by checking that the recorded motion is visually and kinematically consistent with the poses observed in the XR application.

The validation procedure was as follows:

1. **Controlled motion capture.** A short session was recorded while performing simple, easily repeatable motions to facilitate visual comparison.
2. **Dual-engine acquisition.** The same test was executed using both Unity and Unreal Engine integrations, ensuring that the native recording backend is exercised under both runtime environments.
3. **External visualization.** The resulting C3D files were opened using two different C3D-compatible viewers. The recorded trajectories and orientations were compared with what was observed in the XR application during runtime.

The expected outcome is that the pose evolution matches the real-time XR visualization and the C3D reconstruction. In practice, this implies that the coordinate system conversions, unit scaling, and sampling rate metadata are coherent and that there are no systematic

discrepancies. Additionally, the C3D recordings were checked for basic consistency properties: correct session duration, plausible frame count given the configured sampling rate, and absence of discontinuities attributable to the writing process (e.g., periodic missing frames or truncated files).

Pairs of Figures 5.7, 5.8 (Unity) and Figures 5.9, 5.10 (Unreal Engine) present examples of the reconstructed motion for each engine. In each pair, the same hand pose is shown both as an in-game screenshot and as a C3D visualizer view captured after the session. Furthermore, the depicted hand pose corresponds to the one actually performed by the user.

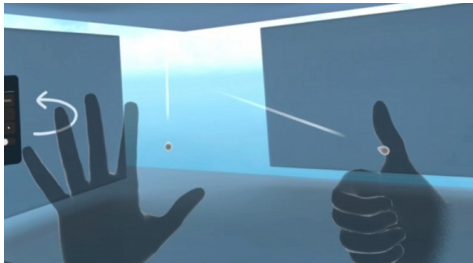


Figure 5.7: Unity Screenshot

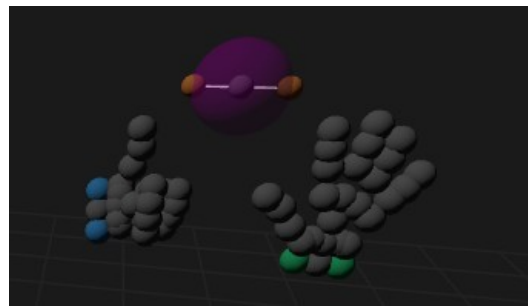


Figure 5.8: C3D-Visualizer



Figure 5.9: UE Screenshot

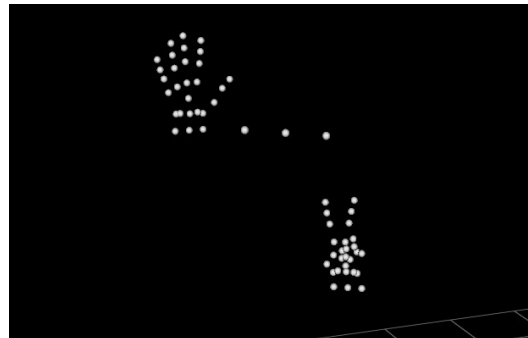


Figure 5.10: Mokka C3D Visualizer

Overall, these experiments confirm that both persistence mechanisms JSON transmission to Supabase and local C3D recording are functional and produce data that remain consistent with the original tracking information generated by the XR runtime.

Conclusions and Future Work

This work has presented the design, implementation, and validation of a cross-platform library for capturing and persisting motion data from virtual reality applications, with a particular focus on biomedical applicability. The proposed solution addresses several limitations inherent to VR-based tracking systems when repurposed as sensing tools for motion analysis, including temporal instability, heterogeneous coordinate systems, and the lack of standardized data persistence mechanisms.

From a technical perspective, the developed library provides a unified interface for motion capture across Unity and Unreal Engine applications, enabling the acquisition of head and hand kinematics in a consistent and configurable manner. Since VR devices are still not as accurate as traditional movement capture systems, special attention has been paid to the temporal aspects of data acquisition, analyzing the impact of different engine update cycles and selecting capture strategies that maximize sampling stability. In addition, the implementation supports multiple data persistence pathways, including structured JSON transmission to external backends and local storage using the C3D format, which remains a reference standard in biomechanical analysis.

Beyond its technical contributions, the relevance of this work lies in its potential impact on biomedical engineering applications. By enabling VR devices to function as flexible motion capture sensors, the proposed approach reduces the barrier to quantitative movement analysis, facilitating its integration into clinical, rehabilitation, and research contexts. The

ability to capture motion data during immersive, task-focused virtual experiences creates new possibilities for evaluating movement behavior under controlled conditions, which are difficult to reproduce in traditional laboratory environments.

The validation experiments carried out in this work show that, despite the inherent limitations of VR systems, coherent and reproducible kinematic data can be obtained when suitable engineering choices are implemented. The visual comparison of motion data exported in C3D format across multiple engines and visualization tools confirms the stability of the captured poses, reinforcing the feasibility of integrating VR-derived data into established biomechanical workflows.

In summary, this work advances the integration of extended reality technologies with biomedical motion analysis by delivering a practical and extensible software framework. While additional research and development are still needed, the outcomes presented here provide a robust foundation for future work. As VR technologies continue to progress, solutions such as the one proposed in this thesis may play a key role in expanding the reach of quantitative movement analysis beyond highly specialized laboratories, promoting more accessible and data-driven methodologies in biomedical engineering.

6.1 Future Work

Although this work focuses on the design and validation of the LibreMotion Capture library, it represents only one component of the broader LibreMotion ecosystem. A natural continuation of this project is the completion and integration of the remaining system components, including the backend infrastructure and data visualization interfaces, in order to provide a fully functional end-to-end platform for biomechanical data management and analysis.

From a technological perspective inside this project, the extension of platform compatibility constitutes a relevant line of future work. The current implementation is limited to Android-based standalone VR headsets using inside-out tracking and an OpenXR runtime. Thus, extending support to standalone devices non Android-based (Apple Vision Pro), PC-based VR systems, WebXR applications, and other XR-capable platforms would increase the flexibility and applicability of the solution. This expansion would require adapting the data acquisition pipeline to different tracking APIs and hardware configurations while preserving a consistent data model.

A particularly relevant direction from a biomedical engineering standpoint is the validation of the proposed system in collaboration with healthcare professionals. Future studies could involve physiotherapists and rehabilitation specialists to assess the usability and clinical relevance of VR-based motion capture during therapeutic exercises. For example, virtual reality devices could be employed as monitoring tools in tele-rehabilitation scenar-

ios, enabling remote supervision of patient movement and objective tracking of functional progress.

In this context, additional developments could focus on remote data transmission and configuration management. A system allowing patients to capture motion data at home and securely transmit C3D files to a clinical backend would facilitate longitudinal follow-up without requiring in-person visits. Similarly, enabling remote configuration of acquisition parameters, such as those defined in the `InitialConfig.json` file, would allow clinicians or researchers to adapt data collection protocols dynamically, improving personalization and experimental control.

Impact of the Project

A.1 Social Impact

The project presented in this work has a potential positive social impact, particularly in the context of healthcare and rehabilitation. By enabling the capture and analysis of human movement using virtual reality devices, the developed library can contribute to the expansion of tele-rehabilitation solutions, allowing patients to perform supervised exercises remotely.

This approach can improve accessibility to physiotherapy services, especially for individuals with reduced mobility, patients living in rural or remote areas, or those with limited access to specialized healthcare facilities. Additionally, the use of widely available consumer virtual reality devices may help reduce existing barriers to access when compared to traditional motion capture systems, which are often costly and limited to specialized laboratories.

Furthermore, the system can facilitate the continuous monitoring of patient progress, allowing more personalized treatments and potentially improving rehabilitation outcomes through data-driven feedback.

A.2 Ethical and Privacy Considerations

The system developed in this project processes biometric data related to human movement, which must be considered sensitive personal information. As such, ethical considerations

regarding data privacy, anonymization, and secure handling are essential.

The proposed solution does not require the storage of personally identifiable information beyond the HMD device data (which, however, could be associated to a person through the HMD vendor), and the captured data can be associated with anonymized session identifiers. This design choice reduces the risk of exposing sensitive personal data while still allowing meaningful biomechanical analysis.

Nevertheless, any application that uses the developed library should comply with applicable data protection regulations, such as the General Data Protection Regulation (GDPR), ensuring informed consent, secure data transmission, and appropriate data storage policies. These considerations are fundamental when dealing with medical or quasi-medical applications involving human subjects.

A.3 Economic Impact

From an economic perspective, the adoption of virtual reality-based motion capture systems may initially increase the cost of physiotherapy services due to the need for specialized hardware and software. However, this increase reflects the current limitations of remote physiotherapy solutions, which often lack accurate and objective movement analysis tools.

In the medium and long term, the proposed approach has the potential to reduce costs by minimizing the need for in-person sessions, optimizing therapist time, and enabling the treatment of a larger number of patients remotely. Additionally, the use of consumer-grade VR devices represents a more affordable alternative to traditional motion capture systems, potentially lowering the overall cost of biomechanical data acquisition.

As a result, the economic impact of the project should be considered in terms of long-term efficiency gains rather than short-term implementation costs.

A.4 Environmental Impact

The environmental impact of the project is generally limited but presents both positive and negative aspects. On the one hand, the use of remote rehabilitation solutions may contribute to a reduction in transportation needs, decreasing carbon emissions associated with patient and professional travel.

On the other hand, virtual reality devices are electronic products whose manufacturing and disposal involve environmental costs. However, since the system is designed to operate on commercially available hardware without requiring specialized equipment, it does not significantly increase the environmental footprint beyond that of existing consumer electronics.

Overall, the reduction in transportation-related emissions may partially offset the envi-

ronmental impact associated with hardware usage, especially when the devices are reused over extended periods.

A.5 Professional Responsibility

Given the potential medical applications of the proposed system, professional responsibility plays a critical role in its design and use. The accuracy and reliability of the captured data is essential, as incorrect or misleading information could negatively affect clinical decisions or rehabilitation outcomes.

Although the system is intended as a support tool rather than a diagnostic device, its use in healthcare contexts requires careful validation, appropriate calibration, and clear communication of its limitations. Healthcare professionals using such tools must ensure that patients understand how to correctly perform exercises and interact with virtual reality applications.

Consequently, the responsibility associated with this project lies not only in the technical correctness of the software but also in its responsible deployment, emphasizing transparency, validation, and user guidance.

Economic Budget

B.1 Tecnological Resources

During the development of the project, two technological devices have been used mainly:

- Workstation (i7-12700KF + RTX 3070 Ti) around 1,500 €.
- Meta Quest 2 (128GB) with accessories around €.

In addition to the hardware resources described above, all the software tools used during the development of this project were based on free and open-source solutions or academic licenses available at no cost. This includes the development environments, programming languages, libraries, game engines, and external dependencies required for implementation and testing. As a result, no additional licensing costs were incurred.

B.2 Human resources

To calculate the cost of the human resources of this project, two main factors are considered: The people/hour time and the Telecommunication Engineer average salary in Spain. The Bachelor Thesis is valued as 12 ECTS, so the employed time must be 360 hours. The average salary of an IT professional in Spain is around 35,000 [25] €, suggesting around 18 €/hour. Thus, the estimated cost of human resources in this project is around 6,480 €.

B.3 Total Cost

The total estimated cost of the project is obtained by combining the technological resources and the human resources involved in its development. Table B.1 summarizes the main cost components considered.

Cost item	Estimated cost (€)
Technological resources	1,800
Human resources	6,480
Total cost	8,280

Table B.1: Estimated total cost of the project

It should be noted that the technological resources used in this project correspond to commercially available consumer devices and general-purpose computing equipment, which may also be used for other academic or professional activities. Therefore, the estimated cost reflects the use of these resources during the development of the project rather than an exclusive investment.

Bibliography

- [1] Pierre Merriaux, Yohan Dupuis, Rémi Boutteau, Pascal Vasseur, and Xavier Savatier. A study of vicon system positioning performance. *Sensors*, 17(7):1591, 2017.
- [2] Matteo Menolotto, Dimitrios-Sokratis Komaris, Salvatore Tedesco, Brendan O’Flynn, and Michael Walsh. Motion capture technology in industrial applications: A systematic review. *Sensors*, 20(19):5687, 2020.
- [3] Fortune Business Insights. 3d motion capture market size, share & global report, 2024.
- [4] Huiyu Zhou and Huosheng Hu. Human motion tracking for rehabilitation-a survey. *Biomedical Signal Processing and Control*, 3:1–18, 2008.
- [5] Steven M. Lavalle, Anna Yershova, Max Katsev, and Michael Antonov. Head tracking for the oculus rift. *Proceedings - IEEE International Conference on Robotics and Automation*, pages 187–194, 9 2014.
- [6] Ana Rojo, Javier Cortina, Cristina Sánchez, Eloy Urendes, Rodrigo García Carmona, and Rafael Raya. Accuracy study of the oculus touch v2 versus inertial sensor for a single-axis rotation simulating the elbow’s range of motion. 26:1651–1662, 2022.
- [7] Jose Angel Santos-Paz, Álvaro Sánchez-Picot, Ana Rojo, Aitor Martín-Pintado-Zugasti, Abraham Otero, and Rodrigo Garcia-Carmona. A novel virtual reality application for autonomous assessment of cervical range of motion: development and reliability study. *PeerJ*, 10:e14031, 9 2022.
- [8] José Angel Santos-Paz, Álvaro Sánchez-Picot, Elena Bocos-Corredor, Filippo Moggioli, Aitor Martin-Pintado-Zugasti, Rodrigo García-Carmona, and Abraham Otero. Neck functional status assessment using virtual reality simulation of daily activities. *Technologies 2025, Vol. 13, Page 248*, 13:248, 6 2025.
- [9] International Data Corporation. Idc - ar & vr headsets market insights.
- [10] Cesar Cadena, Luca Carlone, Henry Carrillo, Yasir Latif, Davide Scaramuzza, Jose Neira, Ian Reid, and John J. Leonard. Past, present, and future of simultaneous localization and mapping: Towards the robust-perception age. *IEEE Transactions on Robotics*, 32:1309–1332, 1 2017.
- [11] The Khronos Group. OpenXR Overview.
- [12] Meta Developers. Create more natural movements using inside-out body tracking and generative legs | meta horizon os developers.
- [13] Meta Developers. Horizon os features overview | meta horizon os developers.

- [14] Meta Developers. Application lifecycle handling | meta horizon os developers.
- [15] Una guía parcial sobre la realidad virtual (de zhihu), 10 2023.
- [16] Meta. Meta quest 2 – technical specifications, n.d.
- [17] Qualcomm Technologies. Snapdragonxr2, 2020.
- [18] Vicon Motion Systems Limited. Plug-in gait reference guide. 2016.
- [19] Game Developer Conference GDP. State of the game industry 2025. Technical report, Game Developer Conference GDP, 3 2025.
- [20] Benjamin Michaud and Mickaël Begon. ezc3d: An easy c3d file i/o cross-platform solution for c++, python and matlab. *Journal of Open Source Software*, 6:2911, 2 2021.
- [21] Claire C Gordon, Cynthia L Blackwell, Bruce Bradtmiller, Joseph L Parham, Patricia Barrientos, Stephen P Paquette, Brian D Corner, Jeremy M Carson, Joseph C Venezia, Belva M Rockwell, Michael Mucher, and Shirley Kristensen. 2012 anthropometric survey of u.s. army personnel: Methods and summary statistics, 2014.
- [22] Eric Lengyel. *3D Math Primer for Graphics and Game Development*. A K Peters/CRC Press, 2nd edition, 2012.
- [23] Luis Llamas. Explanation of nyquist sampling theorem without equations, 2020.
- [24] Supabase. Supabase docker. <https://github.com/supabase/supabase>, 2025. Official Docker-based local development setup.
- [25] Salary: Telecommunications engineer in barcelona, spain 2026 | glassdoor.
- [26] Waqar Ahmad, Hossein Mahdavi, and Ilker Hamzaoglu. An efficient versatile video coding motion estimation hardware. *Journal of Real-Time Image Processing*, 21(2):25, 2024.
- [27] Nadia Baaziz, Dong Zheng, and Demin Wang. Image Quality Assessment Based on Multiple Watermarking Approach. In *2011 IEEE 13th International Workshop on Multimedia Signal Processing*, pages 1–5, 2011.